

Efficient foundation decoders for fault-tolerant quantum computing

Ge Yan,¹ Shanchuan Li,^{1,2} Shiyi Xiao,^{1,3} Pengyue Ma,¹ Hanyan Cao,⁴ Feng Pan,^{4,*} and Yuxuan Du^{1,5,†}

¹College of Computing and Data Science, Nanyang Technological University, Singapore

²Department of Electrical Engineering and Computer Science,

Tokyo University of Agriculture & Technology, Koganei, Tokyo, Japan

³School of Artificial Intelligence, Shanghai Jiao Tong University, Shanghai, China

⁴Science, Mathematics and Technology Cluster, Singapore University of Technology and Design, Singapore

⁵School of Physical and Mathematical Science, Nanyang Technological University, Singapore

Foundation decoders, a class of high-capacity neural decoders, are leading candidates for fault-tolerant quantum computing, with accurate and efficient decoding at large code distances. However, their construction often faces a steep scaling barrier, as larger code distances rapidly amplify the cost of syndrome generation and neural optimization. To address this bottleneck, here we devise neural transfer unification (NTU), a unified framework for efficient foundation decoders. A central feature of NTU is its ability to align decoding tasks across code distances via algebraic structures shared by scalable code families, which enables knowledge learned on smaller codes to accelerate large-scale decoder training. We instantiate NTU as NTU-Transformer, a transformer-based neural decoder tailored for planar surface codes and bivariate bicycle codes. For planar surface codes under circuit-level noise, NTU-Transformer outperforms correlated matching on $[[361,1,19]]$ and extends to $[[625,1,25]]$ with only 27% additional training overhead relative to the transferred baseline. For the bivariate bicycle code with $[[72,12,6]]$, it surpasses Relay-BP in the low-physical-error regime. These results establish our proposal as a scalable route to amortized cross-distance training of foundation decoders for fault-tolerant quantum processors.

🔗 NTU project page: <https://xxxx.github.io/ntu-decoder>

9 INTRODUCTION

Fault-tolerant quantum computation is widely envisioned as the route to unlocking the full promise of quantum computing in the presence of pervasive hardware noise [1–3]. Central to this paradigm is quantum error correction (QEC), which protects logical information by encoding it redundantly across many physical qubits [4–6]. Yet a long-standing challenge in QEC is to develop architectures that are both powerful and practically implementable, owing to the difficulty of balancing robustness, overhead, and compatibility with hardware constraints [7–9]. Among the many proposals developed to date [10–12], planar surface codes [12–14] and highly non-local bivariate bicycle (BB) codes [15] have emerged as leading frameworks, supported by rapid experimental progress across superconducting, trapped-ion and neutral-atom platforms [16–21].

Realizing the potential of these QEC codes requires decoders that are both accurate and efficient, motivating huge efforts to develop decoders across two broad paradigms: conventional and neural approaches. The former, including maximum-likelihood decoding [12, 22–24], matching-based decoders [25–27], and belief propagation [28–30], have driven the progress in QEC, but their scalability remains limited at large code distances. Neural decoders offer a complementary paradigm by learning decoding rules directly from syndrome data [31–34], although early models often lagged behind leading conventional decoders at large code distances. *Foundation*

decoders have transformed neural decoding into a competitive route for practical QEC. Concrete examples are AlphaQubit series [18, 35] and Cascade [36], featured by high-capacity architectures and large-scale training. Despite this progress, current foundation decoders face a severe scaling bottleneck, with computational overheads becoming prohibitive beyond modest code distances. To this end, a key question arises: *can foundation decoders be constructed efficiently beyond this barrier?*

In this study, we provide a positive affirmation by introducing neural transfer unification (NTU), a framework for efficiently constructing foundation decoders via cross-distance transfer. Conceptually, NTU exploits a *structural isomorphism* across code distances, using shared invariances to align different-sized decoding tasks within a common representation space. Attributed to this alignment, NTU can transfer the decoding knowledge learned at smaller distances within the same code family to foundation decoders operating at larger code distances. In this regard, NTU departs from the curriculum learning paradigm of current foundation decoders [37], replacing costly staged training with amortized cross-distance transfer. This paradigm shift turns the foundation decoders from distance-specific training into reusable scaling, substantially reducing computational overhead.

For practical deployment, NTU can be instantiated with diverse neural architectures. We realize this flexibility as NTU-Transformer, a transformer-based foundation decoder designed for high-accuracy cross-distance decoding in planar surface codes and BB codes. Its architecture is built around two specific designs: a new scalable transformer embedding model and a geometry-aware rotary positional encoding, which embed variable-

* feng_pan@sutd.edu.sg

† yuxuan.du@ntu.edu.sg

size syndrome data into a shared latent representation space. Together, these designs preserve the structural isomorphisms shared across code distances while capturing global syndrome correlations, enabling effective cross-distance transfer. We conduct systematic experiments to validate the effectiveness of NTU-Transformer. For surface codes, it outperforms highly optimized correlation-aware matching on the $[[361,1,19]]$ code and reaches the $[[625,1,25]]$ code with only 27% additional training overhead relative to the transferred baseline (see Fig. 1). For the $[[72,12,6]]$ BB code, it also outperforms Relay-BP [38] in the low-physical-error regime. By making these decoders openly available, our work lowers a key computational barrier to high-accuracy neural decoding and provides a practical starting point for real-time, hardware-integrated decoders in fault-tolerant quantum processors.

RESULTS

Structural isomorphism in QEC.— The remarkable efficacy of the proposed NTU transfer learning paradigm stems from a fundamental structural isomorphism. By mapping the intrinsic scale invariance of the physical QEC geometry directly onto the representation space of deep neural networks, we establish a robust framework for distance-agnostic error decoding.

QEC codes and scale invariance A quantum code is conventionally characterized by the standard $[[n, k, d]]$ formalism, denoting the number of physical qubits, logical qubits, and the code distance, respectively. To establish a rigorous foundation for cross-distance transfer, we unify the representation of scalable fault-tolerant code families through the lens of algebraic geometry.

QEC codes. Rather than relying on rigid Euclidean grids, we parameterize the error syndrome space $S \in \{0, 1\}^c$ using a generalized spatiotemporal polynomial coordinate system, where the total number of detectors m scales polynomially with d . Let the coordinate of any syndrome detector be defined by the monomial $v = x^i y^j t^k$, where x , y , and t denote discrete translation operators across the spatial lattice and the temporal syndrome extraction steps. Under this algebraic formalism, the relative topological distance between any two detectors v_1 and v_2 is rigorously defined by their algebraic shift $\Delta v = v_1^{-1} v_2$, independent of the global code boundaries. Furthermore, because the underlying physical parity-check stabilizers are generated by translationally invariant polynomials over $\mathbb{F}_2$, the error propagation dynamics create a highly localized, recurring syndrome correlation graph. Consequently, the localized topological neighborhood $\mathcal{N}(v)$ of any detector—representing the cluster of detectors jointly triggered by local fault mechanisms—exhibits a strictly invariant algebraic geometry:

$$\mathcal{N}(v) = v \cdot \mathcal{M}(x, y, t) \quad (1)$$

where $\mathcal{M}(x, y, t)$ encapsulates the invariant local correlation motif intrinsic to the code family. To demonstrate the concrete universality of this algebraic abstraction, we consider two contrasting paradigms: the geometrically local rotated surface code, whose physical

parity checks are governed by a single spatial generator $g(x, y) = 1 + x + y + xy$ on a bounded 2D lattice, and the high-rate bivariate bicycle (BB) code, constructed over cyclic group rings via an optimized generator pair $A(x, y) = x^3 + y + y^2$ and $B(x, y) = x + x^2 + y^3$ (detailed boundary truncations and group algebraic derivations are provided in Method).

Scale invariance: This algebraic formulation implies that for structured code architectures, the local topology surrounding a detector can be decoupled from global code boundaries. Within a fixed $[[n, k, d]]$ configuration, the invariance of the generating polynomial $P(x, y)$ ensures that bulk detectors share an identical neighborhood indexing system, capturing the spatial invariance of the code lattice. Similarly, because the fault-tolerant syndrome extraction cycle is executed iteratively over successive measurement rounds, this local structure remains strictly conserved over time, establishing a universal temporal invariance.

Crucially, we extend this principle beyond single-code boundaries to formalize a cross-distance scale invariance (as shown in Fig. 1a) for structured code families. When such a code family expands to an arbitrarily large distance, whether through expanding the spatial boundaries of a planar lattice or increasing the algebraic lifting parameters of a quantum LDPC code, the underlying local generator polynomials remain strictly conserved. Under this perspective, the microscopic spatiotemporal motifs optimized on a computationally tractable base code share an identical local geometry with those on unseen, large-scale lattices. Identifying this profound scale invariance provides a powerful physical lens for QEC design, serving as the necessary prerequisite that permits lossless feature extrapolation across dimensions.

Framework of NTU.— To harness the profound scale invariance of quantum codes, the NTU framework introduces a paradigm of structural isomorphism. By embedding the algebraic properties of QEC geometries directly into neural architectures, we unlock distance-agnostic decoding capabilities.

Structural isomorphism. The operational foundation of NTU lies in mapping the invariant algebraic geometry of quantum codes to the latent representation space of deep neural networks. In neural representation learning, feature extraction fundamentally relies on iteratively aggregating information from other nodes. To ensure this process remains code distance-agnostic, NTU dictates that the local feature update $\mathbf{h}_v$ for a detector v at layer l is strictly parameterized over its algebraic neighborhood $\mathcal{N}(v)$ (established in equation 1):

$$\mathbf{h}_v^{(l+1)} = \sigma \left(\sum_{u \in \mathcal{N}(v)} \mathbf{W}_{v^{-1}u} \mathbf{h}_u^{(l)} \right) \quad (2)$$

where σ denotes a non-linear activation, and the learnable weight matrix $\mathbf{W}$ depends exclusively on the relative algebraic shift $v^{-1}u \in \mathcal{M}(x, y, t)$. Furthermore, advanced architectures such as global Transformers extend

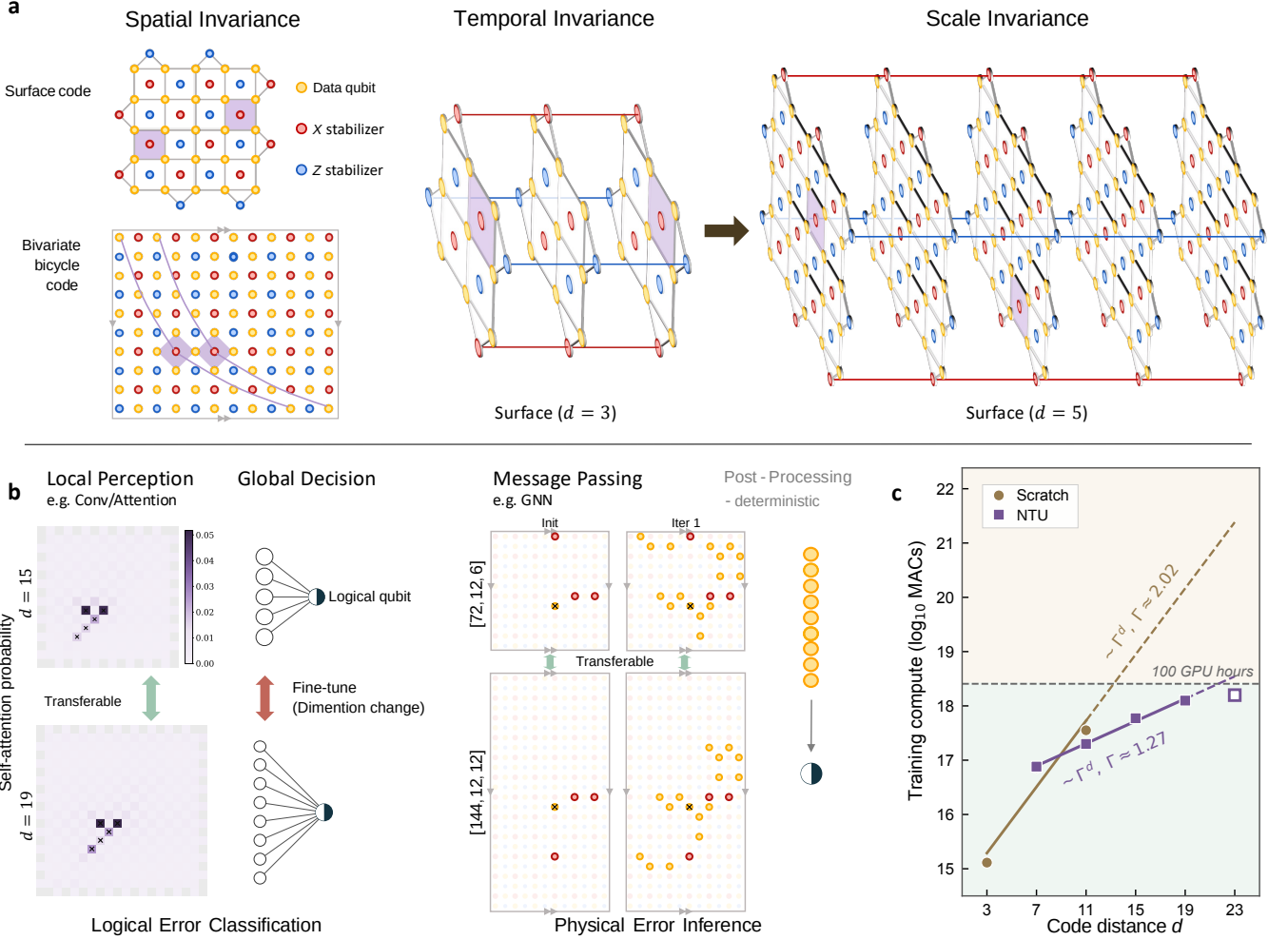


FIG. 1. QEC code invariance and network transfer. **a**, Topological invariances in quantum error correction codes. The fundamental motifs of both planar surface codes and quasi-cyclic bivariate bicycle codes exhibit strict structural isomorphism. This scale invariance manifests across spatial translations, temporal syndrome extraction cycles, and large distance expansions. The shaded regions explicitly denote the invariant local topological neighborhoods defined by the codes' generating polynomials. **b**, Neural decoder network transfer. For logical error classification (left), the architecture is explicitly decoupled into a scale-invariant local perception backbone and a scale-dependent global decision layer. The heatmaps display empirically extracted self-attention probabilities from fully trained NTU-Transformers evaluating the exact same physical error chain on $d = 15$ and $d = 19$ lattices. The perception backbone natively preserves the isomorphic topological motif (highlighted in **a**) without representation collapse. Conversely, physical error inference models (right) bypass learned global decisions entirely. Localized message-passing rules transfer seamlessly to large-scale lattices, followed by deterministic post-processing to resolve the final logical parity. **c**, Computational overhead for scaling. The training compute is evaluated as the threshold required to exceed standard PyMatching accuracy. While train-from-scratch methods suffer from severe cold-start plateaus, the NTU transfer paradigm fundamentally suppresses the computational scaling exponent (reducing Γ from 2.02 to 1.27), enabling highly efficient network generation at massive code distances.

beyond localized patches to capture non-local error correlations. To maintain structural isomorphism in such global paradigms, NTU enforces that any pairwise interaction or attention mechanism between two arbitrary detectors v_1 and v_2 is governed strictly by their relative algebraic distance $\Delta v = v_1^{-1}v_2$. By constraining both localized feature aggregations and global topological interactions entirely through the underlying code polynomials, the neural representation space natively inherits

the physical scale invariance of the QEC system.

Implementation of NTU-Transformer.— Leveraging this structural isomorphism, the neural representation space natively inherits the scale invariance of the QEC geometry, ensuring that latent features learned on small, tractable codes remain valid on extended lattices. For strictly localized paradigms like neural belief propagation, this invariant feature extraction seamlessly yields zero-shot transferability (detailed in Methods).

Conversely, global architectures like Transformers (NTU-Transformer) face the profound challenge of expanding code boundaries, which threatens to distort global attention distributions and re-entangle local features. To preserve transferability, we introduce two highly targeted architectural adaptations. First, the scalable transformer embedding model (STEM) builds accurate local inductive biases by aggregating explicit topological neighbors and categorizes inputs through three algebraic index groups: same-type, cross-type, and temporal predecessor detectors. Second, to prevent the catastrophic collapse of attention maps upon scaling, our geometry-aware rotary position embedding (RoPE) operates strictly on unnormalized relative algebraic shifts Δv . Standard positional encodings often normalize coordinates against the expanding boundaries, which would fatally compress and ruin the previously learned local physical correlations. NTU deliberately avoids this; we rigidly anchor the unnormalized relative spatial phases, forcing the network to perfectly preserve the local correlation motifs while simply allowing the maximum represented distance to expand naturally.

Upon acquiring these scale-invariant representations, the final decoding trajectory diverges into two distinct paradigms (as illustrated in Fig. 1b, detailed in Method section). For physical error inference (PEI), the learned latent features are scattered directly back to the physical data qubits. Because this bipartite mapping is identically governed by the invariant code polynomials, PEI naturally preserves end-to-end zero-shot transferability. In contrast, logical error classification (LEC) requires aggregating these representations for a final global decision. Because the input dimensionality of this global classification layer fundamentally shifts as the underlying code scales, the LEC paradigm seamlessly bridges this specific dimensional gap through highly efficient, few-shot fine-tuning.

Experimental results.—

To rigorously evaluate the NTU framework, we benchmark its decoding performance on the planar surface code using a standard Z -basis memory experiment. We simulate a standard circuit-level depolarizing noise model, where discrete error channels—including initialization, measurement, and all gate operations—are applied with a uniform physical error rate p . Our evaluations span code distances $d \in \{7, 11, 15, 19\}$, executing exactly d syndrome extraction rounds per experiment. To establish a comprehensive baseline, we compare our neural architectures against standard PyMatching, Correlated PyMatching, and the state-of-the-art tensor-network decoder Tesseract. Because exact tensor-network contractions become computationally prohibitive at large scales, Tesseract performance beyond $d = 9$ is rigorously extrapolated to serve as the ultimate heuristic baseline. Decoder performance across all methods is strictly quantified by the logical error per round (LER). For the NTU transfer paradigm, the network is initially trained from scratch on the $d = 7$ base code. Subsequent large-scale

decoders are sequentially initialized via zero-shot parameter transfer from the preceding distance (for instance, transferring the $d = 7$ representation to $d = 11$), restricting the finetuning budget exclusively to realigning the newly expanded boundary conditions.

We first evaluate the baseline efficacy of the NTU framework against these established algorithms. As illustrated in Figure 2a, NTU-Transformer achieves an LER highly competitive with advanced matching-based approaches, tightly approaching the exact Tesseract baseline and Correlated PyMatching at a physical error rate of $p = 0.3\%$. More importantly, the neural decoder demonstrates superior resilience against massive error degeneracy. By binning the decoding events based on the underlying number of detector errors, Figure 2b reveals that while traditional graph-based decoders like PyMatching suffer severe performance degradation in high-error-weight regimes, the neural decoder effectively resolves these complex, highly correlated error configurations. This confirms that the locally parameterized perception mechanism has successfully learned the fundamental physical error dynamics, rather than merely memorizing sparse error syndromes.

The most profound advantage of the NTU paradigm emerges during the training of decoders at large code distances. Training deep neural decoders entirely from scratch presents a severe optimization challenge characterized by a prolonged “cold-start” plateau, where the network temporarily collapses into a trivial optimization minimum before learning meaningful error representations. Crucially, the duration of this dead zone scales drastically with the code distance; as shown in Figure 2c, the accuracy of the $d = 19$ model initialized from scratch remains completely stagnant even after 2,000 training steps. The sequential NTU transfer directly dismantles this scaling barrier. By initializing the network with the scale-independent topological representations transferred from preceding distances, models bypass this initial plateau entirely and achieve an instantaneous vertical liftoff in decoding accuracy. This rapid convergence directly propels the model into the final fine-tuning phase, immediately yielding exceptionally high precision and rendering large-scale neural decoding computationally tractable at a mere fraction of the conventional training cost.

Bivariate Bicycle Code To rigorously demonstrate that our NTU paradigm transcends planar geometries, we extend our evaluation to BB codes—a prominent class of quantum LDPC codes defined by highly non-local, quasi-cyclic hypergraph topologies. Under the identical circuit-level depolarizing noise model, we evaluate the $[[72, 12, 6]]$ and $[[144, 12, 12]]$ architectures, executing exactly $T = d$ syndrome extraction rounds per experiment. We benchmark both the local message-passing (NTU-NeuralBP) and global classification (NTU-Transformer) paradigms against standard BP+OSD and the heavily engineered RelayBP heuristics. For the transfer protocol, models are initially trained on the $[[72, 12, 6]]$ base code and are

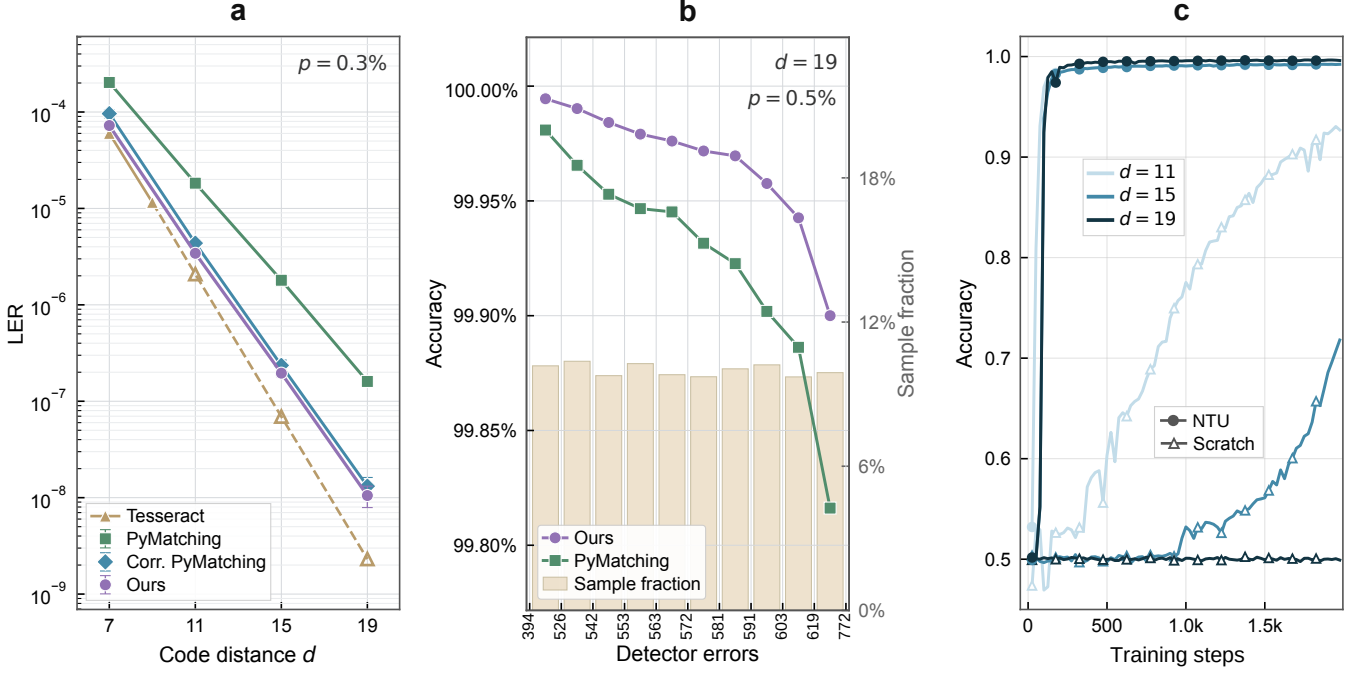


FIG. 2. **Decoding performance and topological transfer dynamics of the neural decoder on the planar surface code.** **a**, Logical error rate as a function of code distance $d \in \{7, 11, 15, 19\}$ evaluated at a physical error rate of $p = 0.3\%$. The transferred base model (purple circles) demonstrates highly competitive fault-tolerant scaling, significantly outperforming standard PyMatching (green squares) and closely approaching the performance of Correlated PyMatching (blue diamonds), establishing a robust baseline for large-distance scaling. **b**, Decoding accuracy resolved by the number of detector errors within the evaluation window on $d = 19$. The underlying histogram denotes the sample fraction distribution across the error weights. While traditional matching-based algorithms experience severe performance degradation in highly degenerate, high-weight error regimes, the neural decoder maintains superior accuracy. This resilience confirms the transferred attention mechanism’s ability to precisely infer complex, highly correlated local physical error dynamics. **c**, Resolution of the cold-start problem via NTU. The plot contrasts the training dynamics (accuracy versus fine-tuning steps) of models trained from scratch (dashed lines) against those initialized via transfer learning (solid lines). Without transfer, models suffer from a prolonged optimization plateau; notably, the $d = 19$ model (dashed purple line) remains completely stagnant at near-zero accuracy even after 2,000 steps. In stark contrast, transferred models exhibit instantaneous vertical convergence. This empirical evidence verifies that the local perception backbone perfectly maps to the topological invariance of the larger lattice, leaving only the global decision layer to quickly adapt to the expanded boundaries.

subsequently transferred to the expanded $[[144, 12, 12]]$ lattice.

As shown in Figure 3a, the decoding efficacies of the two neural architectures diverge significantly. The local NTU-NeuralBP outperforms standard BP+OSD but exhibits a noticeable performance gap when compared to advanced heuristics. In stark contrast, the global NTU-Transformer—utilizing the exact same structural isomorphism architecture deployed for the planar surface code—establishes a highly competitive state-of-the-art on the $[[72, 12, 6]]$ code. It successfully approaches and even surpasses the decoding accuracy of the full RelayBP. This achievement is particularly striking given that RelayBP relies on an overwhelmingly expensive, multi-stage heuristic schedule—typically requiring 80 initial belief-propagation rounds followed by up to 300 sets of 60 relay iterations. By intrinsically learning the underlying physical error dynamics, the Transformer architecture rivals

these heavy heuristic searches with vastly superior computational elegance.

Similar to the phenomena observed in planar surface codes, the global neural architecture exhibits profound resilience against complex error configurations. By evaluating the decoding accuracy across varying detector error weights at a fixed physical error rate of $p = 0.5\%$ (Figure 3b), we observe that traditional belief propagation methods degrade sharply when confronting highly degenerate error syndromes. NTU-Transformer, however, maintains superior decoding stability. This confirms that the dimension-agnostic perception backbone successfully captures the non-local correlation dynamics intrinsic to the quasi-cyclic structure, rather than succumbing to noise degeneracy.

The ultimate stress test of our structural isomorphism lies in scaling the quasi-cyclic boundaries. Figure 3c illustrates the training dynamics when expanding from the

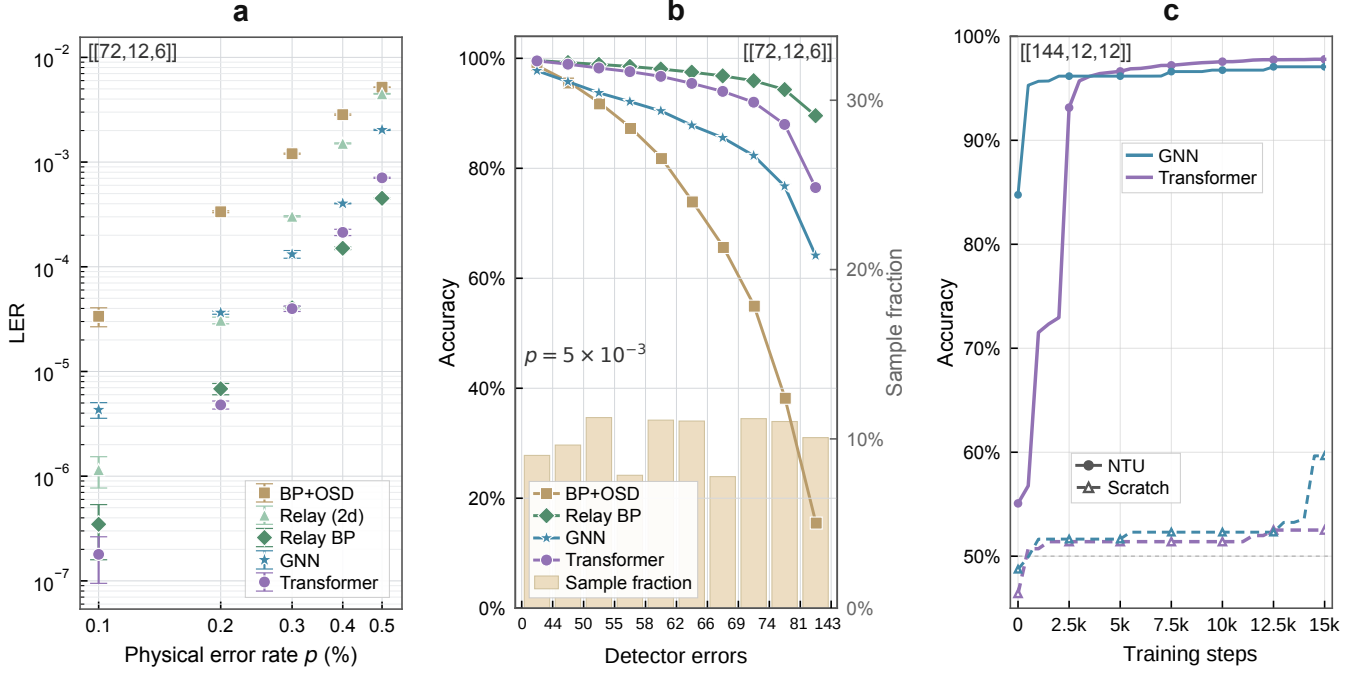


FIG. 3. **Performance evaluation and transfer learning dynamics on non-local BB codes.** **a**, Logical error rates evaluated on the fundamental $[[72, 12, 6]]$ base code. The NTU-Transformer strictly outperforms standard BP+OSD and advanced multi-stage RelayBP variants across all tested physical error rates p . **b**, Decoding accuracy resolved by the number of detector errors at a fixed physical error rate of $p = 0.5\%$. The background histogram indicates the sample fraction for each error-weight bin. While traditional heuristics experience sharp performance drops in the high-detector-error regime due to massive syndrome degeneracy, the neural decoder sustains robust accuracy. **c**, Training convergence on the $[[144, 12, 12]]$ code. Validation accuracy is tracked over the initial training steps to highlight the cold-start phase. Networks initialized from scratch (empty triangles) languish in a prolonged optimization plateau. Conversely, transferring the local perception backbones (solid circles) allows both the NTU-Transformer and NTU-NeuralBP to successfully bypass this initial optimization trap, yielding drastically accelerated convergence.

$[[72, 12, 6]]$ base code to the $[[144, 12, 12]]$ lattice. When initialized entirely from scratch, the expanded cyclic boundary conditions and hypergraph complexity pose a severe optimization challenge. Consequently, both the Transformer and GNN models initialized from scratch are forced into a prolonged optimization plateau, stagnating near random guessing. However, by initializing the networks with the invariant topological representations transferred from the smaller base code, both architectures successfully bypass this massive cold-start phase. By leveraging the pre-trained local algebraic motifs, the networks rapidly realign with the newly expanded cyclic moduli, achieving an instantaneous vertical liftoff in accuracy. These empirical dynamics validate that the algebraic transfer paradigm effectively mitigates the deep learning scalability bottleneck across both planar and quasi-cyclic quantum memory architectures.

DISCUSSION

In this work, we demonstrate that structurally aligning neural networks with the physical invariances of quantum codes fundamentally mitigates the exponential com-

putational barrier that has historically bottlenecked decoder scaling. By explicitly decoupling scale-invariant local perception from scale-dependent global decisions, the NTU framework elegantly bypasses the cold-start optimization plateau, enabling extremely rapid convergence at large code distances. Crucially, we validate that this transferability is not merely a heuristic artifact, but a universal physical principle applicable across diverse topological structures—from planar surface codes to highly non-local qLDPC codes—and across fundamentally distinct decoding architectures.

The practical viability of deep neural networks as fault-tolerant quantum error decoders has fundamentally remained an open question. This skepticism primarily stems from two formidable challenges: the scalability of neural architectures to large code distances, and the stringent microsecond-level latency requirements for real-time decoding. To achieve the execution speeds demanded by physical quantum hardware, the standard deployment trajectory involves aggressively compressing a massive neural decoder into a lightweight architecture. This is typically realized through hardware-aware opti-

395 mizations such as knowledge distillation, low-bit quanti-
 396 zation, and network pruning. However, exploring these
 397 real-time optimization techniques strictly necessitates a
 398 high-quality, full-precision base model to provide the es-
 399 sential structural insights and teacher signals. Histor-
 400 ically, the generation of such large-scale base models
 401 has been severely bottlenecked by massive deep learn-
 402 ing training overheads, paralyzing downstream real-time
 403 explorations.

404 By anchoring our approach in the scale-independent lo-
 405 cal topological invariances of quantum codes, our transfer
 406 learning paradigm effectively mitigates this exponential
 407 computational scaling. We rigorously validate the uni-
 408 versality of this paradigm across diverse quantum mem-
 409 ory topologies (planar surface and quasi-cyclic BB codes)
 410 and distinct decoding architectures (global Transformers
 411 and local GNNs). By successfully generating and pro-
 412 viding these foundational full-precision models, we clear
 413 the most resource-intensive hurdle in the neural decod-
 414 ing pipeline. Ultimately, this work not only resolves the
 415 immediate scalability crisis but also provides the indis-
 416 pensable groundwork required for future developments of
 417 scalable, ultra-fast, and real-time neural decoders.

METHOD

still working on this

Translating the established algebraic invariances into scalable neural decoders requires aligning network architectures directly with the underlying decoding logic. Although neural QEC fundamentally diverges into physical error inference and logical error classification, both paradigms are historically bottlenecked by rigid, scale-dependent network structures. By explicitly anchoring neural perception to the invariant local polynomials, the NTU framework introduces structural transfer mechanisms that overcome this scaling bottleneck across both fundamental routes.

Physical error inference: This decoding paradigm seeks to explicitly identify specific physical errors. Mathematically, it acts as a highly localized mapping $\mathcal{F}_{\text{PEI}} : \{0, 1\}^c \rightarrow [0, 1]^n$, projecting the c -dimensional binary syndrome into marginal error probabilities across the n physical qubits. This formulation inherently reflects the structure of the bipartite Tanner graph, where traditional decoding relies on iterative message passing between the n qubit nodes and c detector nodes. Crucially, the local connectivity and node degrees of this Tanner graph are entirely dictated by the generating polynomial P . As long as P remains invariant during scaling, the local topological properties of the Tanner graph remain structurally identical. However, from an information-theoretic standpoint, inferring exact physical configurations needlessly complicates the optimization landscape, as it actively struggles against quantum degeneracy where massive equivalent error strings yield identical syndromes.

Logical error classification: Contrasting the physi-

cal inference route, classification decoders act as a global mapping $\mathcal{F}_{\text{LEC}} : \{0, 1\}^c \rightarrow \mathcal{P}(L)$, directly projecting the syndrome to a probability distribution over the logical observables $L \in \{I, X, Y, Z\}^k$. This paradigm simplifies the learning objective because it naturally embraces quantum degeneracy, directly evaluating the logical equivalence class rather than tracing individual physical faults. Nevertheless, a severe architectural bottleneck emerges during large-scale scaling. While the input dimension c inflates polynomially with the code distance d , the output dimension k remains rigidly fixed for a given code family. This fundamental dimensional mismatch forces traditional networks to entangle local feature extraction with fixed global boundaries, rendering the entire architecture rigidly scale-dependent. To break this dependency, an ideal scalable architecture must explicitly decouple the scale-invariant local perception phase from the scale-dependent global decision layer (Fig. 1b). Under this decomposed paradigm, the localized physical feature representations become inherently transferable, requiring only the lightweight global decision phase to be briefly fine-tuned to accommodate the expanded boundaries.

METHODS

DATA AVAILABILITY

The data generated and analyzed during the current study are publicly available at the following repositories: Github:

CODE AVAILABILITY

The code used in this study is available at the Github repository

-
- [1] P. W. Shor, Fault-tolerant quantum computation, in *Proceedings of 37th conference on foundations of computer science* (IEEE, 1996) pp. 56–65.
 - [2] A. M. Steane, Efficient fault-tolerant quantum computing, *Nature* **399**, 124 (1999).
 - [3] E. T. Campbell, B. M. Terhal, and C. Vuillot, Roads towards fault-tolerant universal quantum computation, *Nature* **549**, 172 (2017).
 - [4] D. Gottesman, An introduction to quantum error correction and fault-tolerant quantum computation, arXiv preprint arXiv:0904.2557 (2009).
 - [5] B. M. Terhal, Quantum error correction for quantum memories, *Reviews of Modern Physics* **87**, 307 (2015).
 - [6] J. Roffe, Quantum error correction: an introductory guide, *Contemporary Physics* **60**, 226 (2019).
 - [7] P. W. Shor, Scheme for reducing decoherence in quantum computer memory, *Physical Review A* **52**, R2493 (1995).
 - [8] A. M. Steane, Error correcting codes in quantum theory, *Physical Review Letters* **77**, 793 (1996).
 - [9] D. Gottesman, *Stabilizer codes and quantum error correction* (California Institute of Technology, 1997).
 - [10] H. Bombin and M. A. Martin-Delgado, Topological quantum distillation, *Physical review letters* **97**, 180501 (2006).
 - [11] A. Y. Kitaev, Fault-tolerant quantum computation by anyons, *Annals of Physics* **303**, 2 (2003).
 - [12] E. Dennis, A. Kitaev, A. Landahl, and J. Preskill, Topological quantum memory, *Journal of Mathematical Physics* **43**, 4452 (2002).
 - [13] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, Surface codes: Towards practical large-scale quantum computation, *Physical Review A—Atomic, Molecular, and Optical Physics* **86**, 032324 (2012).
 - [14] D. S. Wang, A. G. Fowler, and L. C. Hollenberg, Surface code quantum computing with error rates over 1%, *Physical Review A—Atomic, Molecular, and Optical Physics* **83**, 020302 (2011).
 - [15] S. Bravyi, A. W. Cross, J. M. Gambetta, D. Maslov, P. Rall, and T. J. Yoder, High-threshold and low-overhead fault-tolerant quantum memory, *Nature* **627**, 778 (2024).
 - [16] G. Q. AI, Suppressing quantum errors by scaling a surface code logical qubit, *Nature* **614**, 676 (2023).
 - [17] G. Q. AI and Collaborators, Quantum error correction below the surface code threshold, *Nature* **638**, 920 (2025).

- [18] N. Lacroix, A. Bourassa, F. J. Heras, L. M. Zhang, J. Bausch, A. W. Senior, T. Edlich, N. Shutty, V. Sivak, A. Bengtsson, *et al.*, Scaling and logic in the color code on a superconducting quantum processor, *Nature*, 1 (2025).
- [19] L. Egan, D. M. Debroy, C. Noel, A. Risinger, D. Zhu, D. Biswas, M. Newman, M. Li, K. R. Brown, M. Cetina, and C. Monroe, Fault-tolerant control of an error-corrected qubit, *Nature* **598**, 281 (2021).
- [20] C. Ryan-Anderson, J. G. Bohnet, K. Lee, D. Gresh, A. Hankin, J. P. Gaebler, D. Francois, A. Chernoguzov, D. Lucchetti, N. C. Brown, *et al.*, Realization of real-time fault-tolerant quantum error correction, *Physical Review X* **11**, 041058 (2021).
- [21] D. Bluvstein *et al.*, Logical quantum processor based on reconfigurable atom arrays, *Nature* **626**, 58 (2024).
- [22] S. Bravyi, M. Suchara, and A. Vargo, Efficient algorithms for maximum likelihood decoding in the surface code, *Physical Review A* **90**, 032326 (2014).
- [23] N. Sundaresan, T. J. Yoder, Y. Kim, M. Li, E. H. Chen, G. Harper, T. Thorbeck, A. W. Cross, A. D. Córcoles, and M. Takita, Demonstrating multi-round subsystem quantum error correction using matching and maximum likelihood decoders, *Nature Communications* **14**, 2852 (2023).
- [24] H. Cao, G. Yan, Y. Du, and F. Pan, Maximum likelihood decoding of quantum error correction codes, arXiv preprint arXiv:2605.17230 (2026).
- [25] J. Edmonds, Paths, trees, and flowers, *Canadian Journal of mathematics* **17**, 449 (1965).
- [26] Y. Wu and L. Zhong, Fusion blossom: Fast mwpm decoders for qec, in *2023 IEEE international conference on quantum computing and engineering (QCE)*, Vol. 1 (IEEE, 2023) pp. 928–938.
- [27] O. Higgott and C. Gidney, Sparse blossom: correcting a million errors per core second with minimum-weight matching, *Quantum* **9**, 1600 (2025).
- [28] D. J. MacKay, G. Mitchison, and P. L. McFadden, Sparse-graph codes for quantum error correction, *IEEE Transactions on Information Theory* **50**, 2315 (2004).
- [29] D. Poulin and Y. Chung, On the iterative decoding of sparse quantum codes, arXiv preprint arXiv:0801.1241 (2008).
- [30] H. Yao, W. A. Laban, C. Häger, A. G. i Amat, and H. D. Pfister, Belief propagation decoding of quantum ldpc codes with guided decimation, in *2024 IEEE International Symposium on Information Theory (ISIT)* (IEEE, 2024) pp. 2478–2483.
- [31] G. Torlai and R. G. Melko, Neural decoder for topological codes, *Physical review letters* **119**, 030501 (2017).
- [32] K. Meinerz, C.-Y. Park, and S. Trebst, Scalable neural decoder for topological surface codes, *Physical Review Letters* **128**, 080505 (2022).
- [33] X. Ni, Neural network decoders for large-distance 2d toric codes, *Quantum* **4**, 310 (2020).
- [34] G. Yan, S. Li, and Y. Du, Rethink the role of neural decoders in quantum error correction, arXiv preprint arXiv:2605.12046 (2026).
- [35] J. Bausch, A. W. Senior, F. J. Heras, T. Edlich, A. Davies, M. Newman, C. Jones, K. Satzinger, M. Y. Niu, S. Blackwell, *et al.*, Learning high-accuracy error decoding for quantum processors, *Nature* **635**, 834 (2024).
- [36] A. Gu, J. Ataides, M. D. Lukin, and S. F. Yelin, Scalable neural decoders for practical fault-tolerant quantum computation, arXiv preprint arXiv:2604.08358 (2026).
- [37] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, Curriculum learning, in *Proceedings of the 26th Annual International Conference on Machine Learning* (Association for Computing Machinery, 2009) pp. 41–48.
- [38] T. Maurer, M. Bühler, M. Kröner, F. Haverkamp, T. Müller, D. Vandeth, and B. R. Johnson, Real-time decoding of the gross code memory with fpgas, arXiv preprint arXiv:2510.21600 (2025).
- [39] A. W. Senior, T. Edlich, F. J. Heras, L. M. Zhang, O. Higgott, J. S. Spencer, T. Applebaum, S. Blackwell, J. Ledford, A. Žemgulytė, *et al.*, A scalable and real-time neural decoder for topological quantum codes, arXiv preprint arXiv:2512.07737 (2025).
- [40] S. J. Pan and Q. Yang, A survey on transfer learning, *IEEE Trans. on Knowl. and Data Eng.* **22**, 1345 (2010).
- [41] N. Delfosse and N. H. Nickerson, Almost-linear time decoding algorithm for topological codes, *Quantum* **5**, 595 (2021).
- [42] S. Huang, M. Newman, and K. R. Brown, Fault-tolerant weighted union-find decoding on the toric code, *Physical Review A* **102**, 012419 (2020).
- [43] J. Roffe, D. R. White, S. Burton, and E. Campbell, Decoding across the quantum low-density parity-check code landscape, *Physical Review Research* **2**, 043423 (2020).
- [44] S. Krastanov and L. Jiang, Deep neural network probabilistic decoder for stabilizer codes, *Scientific reports* **7**, 11003 (2017).
- [45] S. Varsamopoulos, B. Criger, and K. Bertels, Decoding small surface codes with feedforward neural networks, *Quantum Science and Technology* **3**, 015004 (2017).
- [46] S. Varsamopoulos, K. Bertels, and C. G. Almudever, Comparing neural network based decoders for the surface code, *IEEE Transactions on Computers* **69**, 300 (2019).
- [47] P. Baireuther, T. E. O’Brien, B. Tarasinski, and C. W. Beenakker, Machine-learning-assisted correction of correlated qubit errors in a topological code, *Quantum* **2**, 48 (2018).
- [48] B. M. Varbanov, M. Serra-Peralta, D. Byfield, and B. M. Terhal, Neural network decoder for near-term surface-code experiments, *Physical Review Research* **7**, 013029 (2025).
- [49] Y.-H. Liu and D. Poulin, Neural belief-propagation decoders for quantum error-correcting codes, *Physical review letters* **122**, 200501 (2019).
- [50] A. S. Maan and A. Paler, Machine learning message-passing for the scalable decoding of qldpc codes, *npj Quantum Information* **11**, 78 (2025).
- [51] A. S. Bhawe, N. Choudhury, and K. Basu, Hypernq: A hypergraph neural network decoder for quantum ldpc codes, arXiv preprint arXiv:2511.01741 (2025).
- [52] H. Cao, F. Pan, Y. Wang, and P. Zhang, qecgpt: decoding quantum error-correcting codes with generative pre-trained transformers, arXiv preprint arXiv:2307.09025 (2023).
- [53] X. Mi and F. Mueller, Toward uncertainty-aware and generalizable neural decoding for quantum ldpc codes, arXiv preprint arXiv:2510.06257 (2025).
- [54] C. Lee, T. Hur, and D. K. Park, Scalable neural decoders for practical real-time quantum error correction, arXiv preprint arXiv:2510.22724 (2025).
- [55] K. Zhang, J. Xu, F. Zhang, L. Kong, Z. Ji, and J. Chen, Latte: A decoding architecture for quantum computing with temporal and spatial scalability, arXiv preprint arXiv:2509.03954 (2025).

- [56] F. Battistel, C. Chamberland, K. Johar, R. W. Overwater, F. Sebastiano, L. Skoric, Y. Ueno, and M. Usman, Real-time decoding for fault-tolerant quantum computing: Progress, challenges and outlook, *Nano Futures* **7**, 032003 (2023).
- [57] N. Liyanage, Y. Wu, S. Tagare, and L. Zhong, Fpga-based distributed union-find decoder for surface codes, *IEEE Transactions on Quantum Engineering* (2024).
- [58] Y. Wu, N. Liyanage, and L. Zhong, Micro blossom: Accelerated minimum-weight perfect matching decoding for quantum error correction, in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (2025) pp. 639–654.
- [59] B. Barber, K. M. Barnes, T. Bialas, O. Buğdaycı, E. T. Campbell, N. I. Gillespie, K. Johar, R. Rajan, A. W. Richardson, L. Skoric, *et al.*, A real-time, scalable, fast and resource-efficient decoder for a quantum computer, *Nature Electronics* **8**, 84 (2025).
- [60] A. Knapen, G. Tao, J. Mack, T. Bruno, M. Saligane, D. Sylvester, Q. Zhang, and G. Subramanian Ravi, Pinball: A cryogenic predecoder for surface code decoding under circuit-level noise, *arXiv e-prints*, arXiv (2025).
- [61] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, A survey of quantization methods for efficient neural network inference, *arXiv preprint arXiv:2103.13630* (2021).
- [62] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, Quantization and training of neural networks for efficient integer-arithmetic-only inference, in *Proceedings of the IEEE conference on computer vision and pattern recognition* (2018) pp. 2704–2713.
- [63] M. Nagel, R. A. Amjad, M. Van Baalen, C. Louizos, and T. Blankevoort, Up or down? adaptive rounding for post-training quantization, in *International conference on machine learning* (PMLR, 2020) pp. 7197–7206.
- [64] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. A. Horowitz, and W. J. Dally, Eie: Efficient inference engine on compressed deep neural network, *ACM SIGARCH Computer Architecture News* **44**, 243 (2016).
- [65] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, How transferable are features in deep neural networks?, in *Proceedings of the 28th International Conference on Neural Information Processing Systems - Volume 2* (MIT Press, 2014) pp. 3320–3328.
- [66] K. He, R. Girshick, and P. Dollar, Rethinking imagenet pre-training, in *2019 IEEE/CVF International Conference on Computer Vision (ICCV)* (2019) pp. 4917–4926.
- [67] A. R. Zamir, A. Sax, W. Shen, L. J. Guibas, J. Malik, and S. Savarese, Taskonomy: Disentangling task transfer learning, in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition* (IEEE, 2018) pp. 3712–3722.
- [68] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, Bert: Pre-training of deep bidirectional transformers for language understanding, in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (2019) pp. 4171–4186.
- [69] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, Language models are few-shot learners, in *Advances in Neural Information Processing Systems*, Vol. 33, edited by H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin (Curran Associates, Inc., 2020) pp. 1877–1901.
- [70] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, *Lora: Low-rank adaptation of large language models* (2021), [arXiv:2106.09685 \[cs.CL\]](https://arxiv.org/abs/2106.09685).
- [71] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, Parameter-efficient transfer learning for NLP, in *Proceedings of the 36th International Conference on Machine Learning*, Proceedings of Machine Learning Research, Vol. 97, edited by K. Chaudhuri and R. Salakhutdinov (PMLR, 2019) pp. 2790–2799.
- [72] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, *et al.*, Segment anything, in *Proceedings of the IEEE/CVF International Conference on Computer Vision* (2023) pp. 4015–4026.
- [73] C. Gidney, Stim: a fast stabilizer circuit simulator, *Quantum* **5**, 497 (2021).
- [74] D. Ruiz, J. Guillaud, A. Leverrier, M. Mirrahimi, and C. Vuillot, Ldpc-cat codes for low-overhead quantum computing in 2d, *Nature Communications* **16**, 1040 (2025).
- [75] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, *Scaling laws for neural language models* (2020), [arXiv:2001.08361 \[cs.LG\]](https://arxiv.org/abs/2001.08361).
- [76] L. Aghababaie Beni, O. Higgott, and N. Shutty, *Tesseract: A search-based decoder for quantum error correction* (2025), [arXiv:2503.10988 \[quant-ph\]](https://arxiv.org/abs/2503.10988).
- [77] P. Panteleev and G. Kalachev, Degenerate quantum ldpc codes with good finite length performance, *Quantum* **5**, 585 (2021).
- [78] T. Müller, T. Alexander, M. E. Beverland, M. Bühler, B. R. Johnson, T. Maurer, and D. Vandeth, Improved belief propagation is sufficient for real-time decoding of quantum memory, *arXiv preprint arXiv:2506.01779* (2025).

FUNDING

AUTHOR CONTRIBUTIONS

COMPETING INTERESTING

The authors declare no competing interests.

CONTENTS

769

770	Method	8
771	References	8
772	A. Preliminary	11
773	1. Surface code	11
774	2. Bivariate Bicycle Codes	12
775	3. Curriculum Learning	12
776	4. Transfer Learning	12
777	B. Literature Review	13
778	1. Traditional QEC decoders.	13
779	2. Neural Decoders	13
780	3. Towards Real-time Neural Decoding	13
781	4. Transfer Learning	14
782	C. Implementation Details of NTU-Transformer	14
783	1. Model Details	14
784	2. Optimization Dynamics and Training Protocols	17
785	D. Implementation details of NTU-NeuralBP and NTU-others	17
786	E. Experimental Details	18
787	1. Evaluation Metrics and Profiling Protocols	18
788	2. Surface Codes	19
789	3. BB Codes	20
790	F. Extended Experimental Results	22
791	1. Training Dynamics of NTU-Transformer on Surface Code	22
792	2. Ablation for NTU-Transformer	23
793	3. Transfer from Partially Trained Source Checkpoints	24
794	Roadmap of this appendix	

795

Appendix A: Preliminary

796

1. Surface code

797 The surface code stands as a prominent example of two-dimensional topological quantum codes, distinguished by its
 798 local stabilizer generators and an exceptionally high threshold against circuit-level noise. In its standard (unrotated)
 799 formulation, data qubits are typically assigned to the edges of a 2D lattice, while the X-type and Z-type parity
 800 checks (stabilizers) correspond to the vertices and plaquettes, respectively [cite]. To optimize hardware resources,
 801 the rotated surface code variant is widely adopted in modern practical implementations [cite]. By effectively rotating
 802 the lattice by 45 degrees, data qubits are instead situated on the vertices, with both X-type and Z-type stabilizers
 803 defined on alternating plaquettes resembling a checkerboard pattern. This rotated architecture significantly improves
 804 the encoding footprint, reducing the physical qubit requirement by nearly a factor of two—requiring exactly d^2 qubits
 805 rather than approximately $2d^2$ to achieve a given code distance d .

806 The strict geometric locality of weight-four stabilizers makes surface codes highly compatible with near-term quan-
 807 tum hardware, particularly planar superconducting qubit architectures. However, despite the constant-factor efficiency
 808 gain provided by the rotated variant, this locality fundamentally restricts the code’s broader encoding capabilities.
 809 Because the total number of physical qubits inherently scales quadratically with the code distance ($N \propto d^2$), the sur-
 810 face code inevitably suffers from a vanishing asymptotic encoding rate. This fundamental limitation ultimately leads
 811 to a substantial physical qubit overhead, posing a major bottleneck for large-scale, practical fault-tolerant quantum
 812 computing.

813 Background of conventional decoders for surface code?

2. Bivariate Bicycle Codes

To overcome the severe overhead limitations of Surface Codes, Bivariate Bicycle (BB) codes have emerged as a highly promising family of quantum low-density parity-check (LDPC) codes. BB codes are algebraically constructed using cyclic shift operators represented by pairs of commuting block-circulant matrices. Specifically, the parity-check matrices are generated from two carefully chosen bivariate polynomials, A and B , which define the X -type and Z -type stabilizers such that the fundamental commutativity condition of CSS codes is strictly guaranteed. This polynomial-based construction naturally yields sparse parity-check matrices while supporting constant finite encoding rates and linear distance scaling ($d \propto N$). Furthermore, the cyclic nature of these generating polynomials endows BB codes with a highly structured quasi-2D graph topology. While lifting strict geometric locality drastically reduces the physical qubit overhead required for fault tolerance, the connectivity of BB codes remains highly regulated. Remarkably, their complex interaction hypergraph can be elegantly decomposed into two locally planar subgraphs. This topological feature makes them exceptionally well-suited for near-term experimental realization, as they can be mapped onto a bi-layer stacked architecture of superconducting qubits using only local intra-layer and direct inter-layer couplings. Consequently, BB codes serve as a crucial bridge between the theoretical efficiency of LDPC codes and the wiring constraints of practical quantum hardware.

Background of conventional decoders for surface code?

3. Curriculum Learning

Curriculum learning is an optimization strategy in machine learning that mimics human learning by presenting training examples in a progressively increasing order of difficulty. Its core mathematical rationale involves smoothing the highly non-convex loss landscape during the initial training phase. By starting with simpler configurations, the neural network is guided toward favorable regions in the parameter space, thereby avoiding poor local minima and facilitating the convergence of complex optimization problems [37].

Within the context of quantum error correction (QEC), curriculum learning has served as a foundational technique for training scalable neural decoders, such as those employing global attention mechanisms [39]. Because the state space of quantum errors expands exponentially with the code distance d and the number of syndrome extraction cycles, directly optimizing a large neural decoder from random initialization is highly inefficient. To address this, curriculum learning protocols typically employ a spatiotemporal scheduler. The training begins with minimal code distances (e.g., $d = 3$ or $d = 5$) and very short temporal windows. As the optimizer stabilizes, the curriculum progressively increases the spatial dimensions and the temporal depth of the error configurations, thereby smoothly transitioning the network to operate on larger code structures.

Do we need this? This part should be in Related Work. If you want to explain how to obtain the yellow line in Figure 1c, move this part to the following section.

4. Transfer Learning

Transfer learning is a pivotal paradigm in deep learning designed to address the challenges of data scarcity and prohibitive training costs. It achieves this by leveraging established knowledge from a source domain $\mathcal{D}_S$ to accelerate convergence and improve generalization in a target domain $\mathcal{D}_T$ [40]. Formally, this process involves the mapping of a learned predictive function from a source task to a target task, significantly reducing the computational resources required compared to optimizing a model entirely from random initialization.

In practice, this knowledge transfer is predominantly executed through parameter-based methodologies and hierarchical feature reuse. Rather than initializing a neural network with chaotic random weights, the target model inherits the optimized parameters from a source model pre-trained on a related task. This allows the target network to commence its optimization trajectory from a semantically rich feature space. Crucially, transfer learning exploits the fact that deep neural networks—particularly convolutional or attention-based architectures—learn hierarchical representations. Lower-level structural motifs, such as local spatial correlations or basic topological features, are often universally applicable across distinct tasks. By freezing or slightly fine-tuning these early perception layers, the network can seamlessly transfer its capability to aggregate local context to the new domain.

Furthermore, when the source and target tasks exhibit mismatched input or output dimensions, cross-architecture adaptation can be seamlessly integrated into the transfer pipeline. Under this regime, the core perception backbone is structurally preserved and transferred, while specific task-dependent modules (such as global decision layers) are structurally modified to accommodate the new dimensional constraints. In the context of quantum error correction, transfer learning paradigms investigate how localized physical error dynamics—learned on a minimal base code or

under a simplified noise model—can be mathematically preserved and reused. By transferring these invariant local topological features, a target neural decoder can bypass the massive computational overhead typically required to construct internal physical representations for large or topologically distinct quantum codes.

Appendix B: Literature Review

1. Traditional QEC decoders.

Traditional QEC decoders for stabilizer codes can be broadly grouped by the graph structures induced by the code family and noise model. The most widely used baseline is minimum-weight perfect matching (MWPM), which reduces decoding to a minimum-weight matching problem solvable via blossom-type algorithms [13, 25], with recent implementations exploiting sparsity and streaming-style dataflow [27]. Union-Find decoders provide an alternative, replacing global optimization with local cluster growth and disjoint-set merging at almost-linear-time complexity [41, 42]. For codes on irregular Tanner graphs, belief propagation (BP) provides a natural iterative framework, though short cycles and code degeneracy hinder naive convergence; augmented variants such as BP+OSD improve robustness at additional computational cost [29, 43]. While these classical decoders offer strong baselines, they generally do not approach maximum likelihood decoding accuracy, particularly under correlated circuit-level noise at large code distances.

2. Neural Decoders

For topological codes whose syndromes admit regular spatiotemporal representations, a range of neural architectures has been explored. Early work applied multilayer perceptrons (MLPs) to flattened syndrome vectors [31, 44, 45]. Convolutional neural networks (CNNs) leverage locality and translation invariance by treating syndrome histories as 2D or 3D volumes [46]. To capture temporal correlations across measurement rounds, sequence models including recurrent networks and temporal convolutional networks (TCNs) have been adopted [47, 48]. Transformer-based architectures further enable long-range interactions and can incorporate soft measurement information [35, 39]. For codes with irregular connectivity, neural belief propagation (NBP) unrolls BP iterations into a trainable network [49], and more general graph neural network decoders learn update rules beyond BP-style messages [50]. Other directions include hypergraph formulations [51], generative modeling [52], and uncertainty-aware decoding [53].

Despite this architectural diversity, a fundamental scalability bottleneck persists: nearly all neural decoders require training from scratch for each code distance, with computational cost growing exponentially in d . State-of-the-art frameworks such as AlphaQubit 2 [39] partially mitigate this through curriculum learning and spatial padding to train a single model up to $d=23$, but extending to any new distance renders prior compute obsolete. Other recent efforts explore scalable architectures [54, 55], yet the exponential training wall remains the central unresolved challenge for large-distance neural decoding.

3. Towards Real-time Neural Decoding

Real-time fault-tolerant QEC imposes stringent microsecond-scale latency constraints on decoders [5, 56], motivating dedicated hardware implementations—often on FPGAs—to sustain streaming syndrome processing within control-cycle budgets. For classical decoders, several hardware-oriented designs have been demonstrated, including distributed Union-Find [57], accelerated MWPM via specialized dataflow [58], clustering-style decoders with predictable latency [59], and cryogenic pre-decoders [60]. Hardware-optimized BP variants have also been mapped to FPGA for irregular Tanner-graph codes [38].

For neural decoders, however, achieving real-time inference requires an end-to-end compression pipeline that bridges the gap between full-precision models and hardware-constrained deployment. This pipeline typically involves three stages: (1) *knowledge distillation* from a high-accuracy teacher model to a compact student, (2) *quantization* to low-bitwidth integer arithmetic (INT8 or INT4) to drastically reduce compute and memory footprint [61–63], and (3) *weight pruning* to exploit sparsity for further acceleration [64]. In our prior work [34], we demonstrated this complete pipeline for surface codes up to $d=9$, showing that INT4 quantization-aware training (QAT) is a *prerequisite*—not a post-hoc optimization—for meeting microsecond-scale latency on FPGAs, and that pruning and distillation provide complementary gains.

No need here.

4. Transfer Learning

The theoretical efficacy of transfer learning stems from the hierarchical nature of neural representations. Seminal empirical research established that the early layers of deep networks capture general features that are universal across disparate datasets, while deeper layers transition to task-specific representations [65]. This discovery catalyzed the widespread adoption of the “Pre-train and Fine-tune” pipeline, which has become the de facto standard in both Computer Vision (CV) and Natural Language Processing (NLP).

In the CV domain, research into rethinking ImageNet pre-training has shown that while training from scratch can eventually achieve comparable accuracy given sufficient data, pre-training provides a significant “jumpstart” in convergence speed and robustness, especially in data-constrained scenarios [66]. Furthermore, the development of task transferability maps, such as *Taskonomy*, has revealed the hidden analytical relationships between different visual objectives, allowing for the strategic reuse of supervision labels across domains [67].

The advent of Transformer-based foundation models has further pushed the boundaries of transfer learning. In NLP, the shift from static word embeddings to context-aware pre-training (e.g., BERT, GPT) enabled models to capture complex linguistic priors that can be transferred to myriad downstream tasks via few-shot learning [68, 69]. However, as model parameters scale into the billions, the computational cost of full-parameter fine-tuning has become a new “computational wall”.

To mitigate this, the literature has increasingly focused on Parameter-Efficient Fine-Tuning (PEFT). Methodologies such as Low-Rank Adaptation (LoRA) demonstrate that by freezing the pre-trained backbone and optimizing a minimal set of auxiliary low-rank matrices, one can achieve state-of-the-art performance with a fraction of the trainable parameters [70]. Similarly, the use of bottleneck adapters allows for modular knowledge insertion without compromising the integrity of the original foundational features [71].

Pushing the boundaries of transferability, the most recent paradigm shift involves transfer without gradient updates, colloquially known as In-Context Learning (ICL). By leveraging the immense parametric memory of large language models, ICL frames downstream adaptation purely as a conditioning problem, inferring task distributions dynamically from hidden state activations. Parallel to this, the vision community has achieved analogous zero-shot transfer capabilities through universal segmentation engines like the Segment Anything Model (SAM) [72].

Appendix C: Implementation Details of NTU-Transformer

1. Model Details

Our foundational neural architecture largely builds upon the recurrent-Transformer hybrid design principles established by AlphaQubit 2 [39]. To achieve seamless scalability, the architecture is explicitly decoupled into a localized spatiotemporal perception backbone and a global decision readout. Detailed implementation information is provided in Table I.

TABLE I. Model architecture and training hyperparameters for the NTU decoding framework. The parameter configurations reflect the distinct topological paradigms of the global classification and local inference models.

Configuration Module	NTU-Transformer	NTU-NeuralBP
<i>Architecture Specifications</i>		
Core Dimensions	$d_{\text{model}}=512, n_{\text{heads}}=8$	$d_{\text{hidden}}=64$
Network Depth	5 GRU layers, 6 TF layers	Up to $2d$ BP iterations (Curriculum)
Feed-Forward / MLP Size	SwiGLU $d_{\text{hidden}}=1024$	3-layer MLP $d_{\text{hidden}}=64$
Readout Mechanism	2-layer Cross-Attention ($d_{\text{FFN}}=2048$)	MLP with GELU
Positional Encoding	Geometry-Aware RoPE ($\Theta=100$)	None required
Stabilization Layers	RMSNorm	LayerNorm, ScaledTanh
Total Parameters	$\sim 32\text{M}$	$\sim 50\text{K}$
<i>Training Dynamics</i>		
Peak Learning Rate	2×10^{-5}	3×10^{-4}
Effective Batch Size	4096 (via accumulation)	Target 2048 (Base code)
Weight Decay	10^{-2} (Backbone only, Readout $\lambda=0$)	10^{-4}
Gradient Clipping (Max Norm)	1.0	1.0
Regularization	Stabilizer dropout ($p_{\text{drop}}=0.5$, annealed)	–
Loss Formulation	BCE + Classical Distillation (β annealed)	Focal Loss + Syndrome Consistency ($\lambda=0.2$)

944 *a. Topological Input Embedding.* Unlike hardware-specific decoders (e.g., AlphaQubit) that rely on continuous
 945 analog readouts to identify leakage and soft errors, our framework operates strictly on the more challenging regime of
 946 pure binary detector events. Processing highly sparse binary inputs directly via standard linear projections typically
 947 destabilizes deep network optimization, as the overwhelming prevalence of zeros naturally triggers gradient norm
 948 explosions and disrupts global norm clipping strategies.

949 To resolve this optimization bottleneck, we map the raw binary events into a dense, continuous latent space via
 950 discrete topological lookup tables. Crucially, to preserve the transferability of the NTU paradigm, the vocabulary
 951 of these lookup tables is rigidly defined by the local invariant connectivity motif of the specific code family, rather
 952 than arbitrary matrix unrolling. For planar surface codes, this topological motif is geometrically straightforward.
 953 We construct a spatial embedding by concatenating the binary value of a detector with its four same-type diagonal
 954 neighbors. Concurrently, a cross-type hint embedding captures the four orthogonal opposite-type neighbors, and a
 955 temporal embedding encodes parity shifts across adjacent time steps.

956 Adapting this topological embedding to non-local Bivariate Bicycle (BB) codes requires a fundamental paradigm
 957 shift. Because hypergraphs lack a native planar geometry, defining local neighbors via arbitrary large metrics (such
 958 as torus L2 distances) inevitably breaks the structural isomorphism when the global code dimensions ($L \times M$) scale.
 959 To circumvent this, we formulate a *polynomial-induced* topological embedding. By analytically deriving the physical
 960 neighbor offsets directly from the code’s generating cyclic shift polynomials—specifically extracting the off-diagonal
 961 terms of $A \cdot A^T$ and $B \cdot B^T$ for same-type interactions, and $A \cdot B$ for cross-type interactions—we guarantee that the
 962 local neighborhood structure remains strictly identical regardless of the code distance. Consequently, the embedding
 963 vocabulary perfectly aligns the input feature space across different quasi-cyclic architectures (e.g., $d = 6$ and $d = 12$)
 964 without requiring parameter reallocation. These spatially and cross-type constructed embeddings are then summed
 965 with a temporal sinusoidal positional encoding and processed through a stem residual block, forming a robust, scale-
 966 invariant input representation $\mathbf{E} \in \mathbb{R}^{B \times T \times N \times d_{\text{model}}}$.

967 Crucially, this topology-driven embedding strategy is not merely an ad hoc engineering adaptation for BB codes,
 968 but rather a unified and universal paradigm. In retrospect, the spatial embedding constructed for the planar surface
 969 code represents a highly localized, special case of this exact principle, where the algebraic stabilizer interactions
 970 naturally degenerate into strict 2D nearest-neighbor spatial offsets. By defining the input embedding vocabulary
 971 strictly through fundamental stabilizer intersection rules—whether formalized via geometric chain complexes for
 972 topological codes or generating polynomials for qLDPC codes—this methodology fundamentally transcends specific
 973 code families. Consequently, this algebraic embedding framework is intrinsically generalizable to a broader spectrum
 974 of advanced quantum error-correcting architectures. Looking forward, this scale-invariant approach can be seamlessly
 975 extended to color codes (by mapping multi-colored plaquette intersections) or generalized lifted product codes, laying
 976 an architecture-agnostic input foundation for truly universal neural decoding.

977 *b. Spatiotemporal Perception Backbone.* Rather than treating the error syndrome as a monolithic three-
 978 dimensional block code—which computationally entangles spatial parity checks with temporal fault tolerance—our
 979 architecture explicitly decomposes the decoding process into spatial and temporal dimensions. While this spatiotem-
 980 poral decoupling inherently relaxes the rigorous 3D fault-tolerant block structure, we adopt it because it drastically
 981 reduces computational complexity and uniquely enables the topological transferability of the spatial perception mod-
 982 ule.

983 Given a code with N check nodes (stabilizers) observed over T measurement rounds, the backbone interleaves
 984 lightweight recurrent units with spatial Transformer blocks. The network follows a structured sequence: $\text{RNN} \rightarrow$
 985 $\text{RNN} \rightarrow [\text{TF} \times 2] \rightarrow \text{RNN} \rightarrow [\text{TF} \times 2] \rightarrow \text{RNN} \rightarrow [\text{TF} \times 2] \rightarrow \text{RNN}$, comprising 5 Gated Recurrent Unit (GRU) layers
 986 and 6 Transformer layers. At each time step t , the per-stabilizer GRU cells propagate latent error representations
 987 along the temporal axis, while the Transformer blocks perform full spatial self-attention over all N stabilizers. Each
 988 Transformer block employs a SwiGLU feed-forward network. The model is configured with a hidden dimension
 989 of $d_{\text{model}}=512$, $n_{\text{heads}}=8$, and a SwiGLU intermediate dimension of 1024, yielding approximately 32M parameters.
 990 Because this backbone strictly models relative spatial and temporal correlations, its weights are fully scale-independent
 991 and perfectly transferable across different code distances.

992 *c. Global Decision via Cross-Attention.* The readout stage faces a severe dimensional challenge: it must compress
 993 the evolving $\mathcal{O}(N)$ spatial detector representations into a fixed-dimensional prediction for the k logical qubits. Tra-
 994 ditional pooling mechanisms (such as mean or max pooling) inadvertently destroy the fine-grained, highly correlated
 995 spatial parity information essential for resolving massive error degeneracy.

996 To circumvent this information loss, we employ a cross-attention mechanism for the global decision phase. The
 997 model introduces learnable logical query tokens that cross-attend to all N stabilizer representations at the final time
 998 step. This design acts as an advanced, geometry-aware pooling mechanism, granting the decision layer a global
 999 receptive field while optimally aggregating distributed parity evidence. Furthermore, this cross-attention projection is
 1000 structurally independent of the input sequence length N , rendering the global decision layer theoretically transferable.
 1001 However, as the code distance scales, the sheer increase in the number of participating detectors N systematically

shifts the variance and scale (norm) of the query-key-value (QKV) attention logits. Consequently, while the structural mapping remains intact, this global decision layer mandates a brief fine-tuning phase to recalibrate the attention normalizations and realign the lo

d. Geometry-Aware RoPE Standard Transformer positional encodings—whether learned or index-based sinusoidal—rely on sequence positions. This paradigm catastrophically fails during topological transfer because scaling the code distance exponentially alters the sequence length N , fundamentally destroying pre-learned spatial relationships. To guarantee structural isomorphism, we implement a geometry-aware RoPE that operates strictly on the intrinsic algebraic coordinates of the quantum codes, completely devoid of learned positional biases or distance-specific constraints.

Crucially, the bedrock of our RoPE design is the strict use of unnormalized coordinates. Normalizing coordinates by the boundary dimensions would continuously compress the local relative distances as the code scales, breaking transferability. For planar surface codes, we directly encode the absolute physical lattice coordinates (p_x, p_y) . Consequently, the relative displacements $(\Delta p_x, \Delta p_y)$ between neighboring stabilizers remain strictly invariant—generating the exact same RoPE phase shifts—regardless of the global code distance d .

Furthermore, this coordinate assignment is not restricted to planar topologies but emerges as a specialized planar instance of a more universal algebraic framework. For highly non-local BB codes, we directly utilize the discrete matrix coordinates (l, m) originating from their block-circulant generating polynomials. Because we explicitly avoid any spatial normalization (e.g., dividing by the torus dimensions L or M), the relative matrix coordinate offsets corresponding to the local multi-body interactions remain absolutely identical whether the base code is at $d = 6$ or the transferred code is at $d = 12$. Thus, the same RoPE machinery perfectly aligns the input space across both geometric and algebraic code families.

To ensure this unnormalized coordinate mapping supports seamless transfer learning, we tailor the RoPE base frequency ($\Theta = 100$) to match the discrete coordinate scale of quantum parity checks, rather than adopting conventional NLP default values ($\Theta = 10,000$). This spectral calibration enforces a vital functional dichotomy: the high-frequency channels rotate rapidly to capture the strictly invariant local neighborhood structures (relative angles), while the low-frequency channels remain monotonic across the entire lattice extent. When transferring to a large-scale code, the invariant high-frequency local attention maps are preserved flawlessly. Concurrently, the monotonic low-frequency embeddings gracefully extrapolate to the newly expanded boundaries without inducing discontinuous phase wrapping, allowing the global decision layer to swiftly adapt during fine-tuning.

e. Process Supervision via Classical Distillation. Training the decoupled spatiotemporal architecture using exclusively the final logical parity induces severe optimization bottlenecks at large distances. The combination of prolonged recurrent gradient paths (where $T \propto d$) and the extreme sparsity of binary parity labels—which ambiguously conflate “zero errors” with “cancelled errors”—routinely traps the network in a trivial, all-zeros optimization minimum.

While circuit simulators can theoretically extract exact intermediate logical observables by accessing the underlying quantum state vector, such operations are fundamentally impossible on physical quantum hardware. To ensure our training paradigm remains strictly applicable to experimental device data, we employ a hardware-compatible classical distillation strategy. At each intermediate time step t , we construct a cumulative partial syndrome by masking all future detection events beyond t . We then query a reliable classical heuristic (e.g., an MWPM decoder) with this truncated syndrome. The resulting pseudo-label represents the classical decoder’s best estimate of the logical parity if the quantum experiment were abruptly terminated at round t .

This generates a dense, step-by-step supervision signal. The network is subsequently trained using a composite objective: $\mathcal{L} = \mathcal{L}_{\text{final}} + \beta(s) \cdot \mathcal{L}_{\text{process}}$, where $\mathcal{L}_{\text{final}}$ evaluates the cross-entropy against the true final ground-truth observable, and $\mathcal{L}_{\text{process}}$ evaluates the predictions against the intermediate MWPM pseudo-labels. Crucially, the distillation coefficient $\beta(s)$ is dynamically annealed over the training steps s . During the early training phase, β is set exceptionally high to flood the deep recurrent layers with strong, deterministic gradient signals, thereby rapidly dismantling the cold-start plateau.

However, as training progresses, β is systematically decayed to a minimal residual value. This relaxation is theoretically vital. Because the classical teacher (e.g., MWPM implemented via PyMatching) prioritizes minimum-weight paths and inherently lacks true maximum-likelihood degeneracy awareness, forcing the neural network to strictly mimic its outputs indefinitely would impose an artificial performance ceiling. By diminishing the teacher’s influence in the late training stages, the neural decoder is forced to optimize directly against the true logical outcomes, thereby learning to independently resolve complex, highly correlated error manifolds. This capability to transcend the distillation baseline is empirically validated in Figure 2a and 2b; although the network utilizes PyMatching pseudo-labels to bootstrap its intermediate representations, the fully trained neural decoder ultimately achieves logical error rates strictly superior to the PyMatching algorithm itself, particularly in high-weight, massive-degeneracy regimes.

2. Optimization Dynamics and Training Protocols

- a. Online Generation and Stochastic Regularization.* To prevent the neural decoder from merely memorizing sparse error manifolds, we employ a strict online data generation strategy via the Stim circuit simulator [73]. At each training iteration, batches of syndrome samples are dynamically drawn from the underlying circuit-level noise models. Because the quantum state space expands exponentially, this on-the-fly generation ensures maximal sample diversity and completely eliminates classical overfitting. To further regularize the perception backbone, we apply stochastic stabilizer dropout [39]. We find that a precise annealing schedule is critical to resolving the asymptotic error tails at large distances: dropout is maintained at full strength ($p_{\text{mask}}=0.8$, $p_{\text{drop}}=0.5$) for the first 50% of training, linearly decayed to zero by 80%, and completely disabled for the final 20%. This allows the network to initially learn highly robust, distributed parity representations, before utilizing its full deterministic precision to resolve the final fractions of the logical error rate.
- b. Selective Regularization and Optimization.* The network is optimized using AdamW with a cosine learning rate schedule, a 100-step linear warmup, and global gradient clipping (max norm 1.0) to prevent instability from the sparse binary inputs. Crucially, we enforce a highly selective weight decay strategy ($\lambda = 10^{-2}$). While weight decay is applied to the core linear projection weights (QKV, SwiGLU, and GRU kernels), it is strictly excluded from all bias terms, RMSNorm/LayerNorm scales, and embedding tables. Most importantly, the entire global decision readout—including the cross-attention weights and the final classification head—is exempt from decay. This selective preservation is a deliberate physical choice: at the tail end of training, weight decay would artificially damp the high-precision logits required to confidently resolve massive error degeneracies, prematurely freezing the optimization and degrading convergence.
- c. Transfer Protocol and Hardware Configuration.* The NTU scaling ladder progressively transfers models from minimal base codes to large lattice sizes. During a transfer phase (e.g., from distance d' to $d > d'$), the scale-invariant learnable parameters—comprising the topological embeddings, the recurrent perception backbone, the Transformer weights, and the global decision layers—are transferred identically. Conversely, all dimension-dependent auxiliary buffers (such as absolute coordinate mappings, gather indices, and validity masks) are explicitly discarded and re-initialized to match the expanded boundary conditions. The optimizer state is reset, and a new cosine warmup-decay cycle is initiated to smoothly realign the global decision layer. All experiments were conducted in bfloat16 mixed precision on NVIDIA H800 GPUs using PyTorch Distributed Data Parallel (DDP). Detailed hyperparameters are summarized in Table I.

Appendix D: Implementation details of NTU-NeuralBP and NTU-others

- Unlike the Transformer, which requires explicit spatiotemporal decoupling and coordinate-based relative positional encodings to perform global logical classification, the NTU-NeuralBP operates directly on the intrinsic bipartite Tanner graph of the quantum code. This paradigm bypasses the logical abstraction entirely, executing localized physical error inference. By natively adopting the graph’s topology, structural isomorphism and parameter transferability are fundamentally guaranteed without coordinate engineering.
- a. Bipartite Perception Backbone and Stabilization.* The neural message-passing architecture strictly mirrors the fundamental Tanner graph connectivity, defining iterative updates between check nodes (stabilizers) and variable nodes (data qubits). However, traditional Belief Propagation (BP) algorithms—which rely on min-sum or sum-product approximations—frequently suffer from severe message oscillations and convergence failures when applied to topological codes due to the abundance of short, degenerate loops.
- To overcome this, NTU-NeuralBP completely parameterizes the message calculation and state updates. During each variable-to-check and check-to-variable pass, messages are constructed using non-linear Multi-Layer Perceptrons (MLPs) rather than rigid algebraic rules. Crucially, the node state updates are governed by Gated Recurrent Units (GRUCells). The inherent memory gating mechanism of the GRU allows the network to automatically recognize repetitive message cycles and actively damp loop-induced oscillations. Furthermore, to prevent the feature variance explosions common in deep iterative networks, we introduce a **ScaledTanh** activation—equipped with a learnable, strictly clamped scale parameter—combined with **LayerNorm**. This custom stabilization forces the latent log-likelihood ratios to remain well-behaved even when the iteration depth expands.
- b. Physics-Informed Readout and Loss Formulation.* Because the physical error inference task operates on individual data qubits, the prediction target exhibits extreme class imbalance (at low physical error rates, the vast majority of qubits experience no errors). To prevent the optimization from collapsing into a trivial “no-error” prior, we supervise the model using a Focal Loss ($\alpha = 0.95, \gamma = 2.0$).

Furthermore, to embed strict topological awareness into the gradient landscape, we incorporate an auxiliary Syndrome Consistency Loss ($\lambda = 0.2$). This loss explicitly maps the predicted continuous error probabilities back onto the parity-check matrix, penalizing network outputs that violate the physically observed initial syndrome. This physics-informed loss directly guides the neural decoder toward physically valid error manifolds.

c. Depth Curriculum and Native Transferability. Deep message-passing networks frequently exhibit severe initial instability. To stably train the NTU-NeuralBP, we employ an iterative depth curriculum. The model is initially trained to execute shallow message-passing trajectories (e.g., $N_{\text{iter}} = 8$). As the validation accuracy plateaus, the curriculum dynamically unlocks deeper unrollings (up to $N_{\text{iter}} = 2d$). This allows the network to first master local error resolution before progressively learning complex, long-range loop neutralizations.

Because the NTU-NeuralBP’s parameter space ($\sim 50\text{K}$ parameters) is entirely restricted to the localized node-update and message-generation MLPs, it is strictly invariant to the global graph size. When transferring from a minimal base code to a large lattice, the exact trained weights are loaded seamlessly; the only operational change is the substitution of the sparse edge index matrices to reflect the expanded topology. This native scaling property validates the GNN as the purest instantiation of topological transfer learning in quantum error correction.

Appendix E: Experimental Details

1. Evaluation Metrics and Profiling Protocols

a. The Calculation of LER per Round and per Qubit To compare decoding performance across different experiment lengths and different code families, we extract the logical error rate (LER) both per round and per qubit. In this subsection, “per qubit” always refers to *per logical qubit*. Here, one round is taken to mean one full QEC cycle. Let $P_{\text{block}}(R)$ denote the block-level logical error probability after R rounds of syndrome extraction. For a single logical qubit, we define the logical fidelity as

$$F(R) = 1 - 2P_{\text{block}}(R). \quad (\text{E1})$$

Assuming independent round-by-round accumulation of logical errors, the fidelity satisfies

$$F(R) = (1 - 2\epsilon_{\text{round}})^R, \quad (\text{E2})$$

where ϵ_{round} is the LER per round. Thus,

$$\epsilon_{\text{round}} = \frac{1}{2} \left[1 - (1 - 2P_{\text{block}}(R))^{1/R} \right]. \quad (\text{E3})$$

If results at multiple round numbers are available, we instead fit

$$\log F(R) = \log F_0 + R \log(1 - 2\epsilon_{\text{round}}), \quad (\text{E4})$$

and extract ϵ_{round} from the fitted slope. This logical-fidelity-based extraction is consistent with the LER definition adopted in AlphaQubit 2 and related QEC studies [16, 17, 39].

For a code encoding k logical qubits, we further define the block-level logical error probability per logical qubit as

$$P_{\text{LQ}}(R) = 1 - (1 - P_{\text{block}}(R))^{1/k}. \quad (\text{E5})$$

This normalization converts the total block-level logical error probability into the effective logical error probability of a single logical qubit, and is introduced here to enable fair comparison between code families with different logical encoding rates [74].

The corresponding LER per logical qubit per round is

$$\epsilon_{\text{LQ}} = \frac{1}{2} \left[1 - (1 - 2P_{\text{LQ}}(R))^{1/R} \right]. \quad (\text{E6})$$

Unless otherwise stated, the LER per qubit reported in this work refers to ϵ_{LQ} .

In the low-error regime, $P_{\text{block}}(R) \ll 1$, this reduces to

$$P_{\text{LQ}}(R) \approx \frac{P_{\text{block}}(R)}{k}, \quad \epsilon_{\text{LQ}} \approx \frac{P_{\text{block}}(R)}{kR}. \quad (\text{E7})$$

This approximation is useful for intuition, while all reported values in this work are computed using the exact expressions above.

1144 *b. Methodology for Profiling Complexity and Throughput.* To rigorously evaluate the computational overhead of
 1145 the NTU framework, we employed a dual-metric profiling methodology. First, to assess theoretical computational
 1146 complexity, we quantified the forward Multiply-Accumulate Operations (MACs) using the `fvcore` library. By profiling
 1147 the model in `train()` mode—thus strictly capturing the overhead of all intermediate process supervision branches—
 1148 we estimate the total computational cost per training step as $3\times$ the forward MACs, aligning with established scaling
 1149 law heuristics [75].

1150 Second, to measure real-world empirical throughput, we conducted rigorous wall-clock profiling. Because PyTorch
 1151 executes CUDA operations asynchronously, naive time-stamping yields highly distorted measurements. To eliminate
 1152 these inaccuracies, we inserted hardware-level synchronization barriers (`torch.cuda.synchronize()`) around the
 1153 critical execution blocks. This precision allowed us to explicitly decouple a single training step into three distinct
 1154 components: the CPU-bound online data generation and preparation phase, the GPU-bound pure forward-backward
 1155 computation phase, and the Distributed Data Parallel (DDP) communication and optimizer update phase. By
 1156 isolating the pure computation time, we ensure a fair and architecturally focused comparison across different code
 1157 distances and baseline algorithms.

2. Surface Codes

1159 *a. Parameters Setting* To ensure robust performance and efficient scaling, our model employs a highly cus-
 1160 tomized training strategy tailored to the surface code environment. During progressive distillation, the Correlated
 1161 PyMatching teacher extracts a Detector Error Model (DEM) with decomposed errors (`decompose_errors=True`) to
 1162 generate high-fidelity Y_{fake} targets. The reliance on these intermediate targets is carefully modulated: a dynamic
 1163 teacher-forcing factor (β) decays from 8.0 to 0.2 to smoothly transition the model from process-guided learning to
 1164 ground-truth supervision, alongside time-progressive weights (w_t) that appropriately emphasize later decoding steps.
 1165 To prevent early-stage mode collapse and overfitting, a structural dropout schedule progressively decays the stabi-
 1166 lizer mask probability (from 0.8) and drop ratio (from 0.5) to zero by the late stages of training. Optimization is
 1167 driven by the AdamW algorithm utilizing mixed-precision (`bfloat16`) computations, a 100-step linear warmup, and
 1168 a cosine annealing schedule that smoothly decays the learning rate to 5% of its base value. Crucially, a decoupled
 1169 `weight_decay` strategy applies a targeted 10^{-2} penalty exclusively to structural projection weights, while strictly by-
 1170 passing (`weight_decay=0.0`) biases, normalization layers, discrete spatial/temporal embeddings, and logical readout
 1171 queries to preserve the network’s representational capacity. Finally, the computationally intensive online MWPM
 1172 label generation is seamlessly managed via Distributed Data Parallel (DDP) with high-efficiency multi-process data
 1173 prefetching.

TABLE II. Experimental Parameter Configuration for Different Distances

Distance (d)	Rounds (r)	Initial Learning Rate	p_{train}	p_{eval}	Local Batch Size	Max Steps
3	3	1e-4	0.005	0.005	64	100000
7	7	2e-5	0.005	0.005	32	100000
11	11	2e-5	0.005	0.005	16	150000
15	15	2e-5	0.005	0.005	8	200000
19	19	2e-5	0.005	0.005	4	250000
23	23	2e-5	0.007	0.007	2	150000
25	25	2e-5	0.007	0.007	2	150000

1174 *b. Baseline Methods* We benchmark the NTU-Transformer against three non-trainable decoding baselines for the
 1175 rotated surface code: standard PyMatching, Correlated PyMatching, and Tesseract. These baselines span the most
 1176 relevant accuracy-complexity spectrum for circuit-level surface-code decoding: a fast graphlike MWPM decoder, a
 1177 stronger correlation-aware matching decoder, and a search-based most-likely-error reference.

1178 *Common evaluation protocol.* All baseline decoders are evaluated on the same held-out detector-event samples
 1179 used for the neural decoder. For each distance d , we generate a rotated surface-code memory circuit with $r = d$
 1180 syndrome-extraction rounds under the same circuit-level depolarizing noise model used in Fig. 2. For the LER curves
 1181 and the detector-error-binned accuracy analysis, the physical error rate is the one specified in the corresponding panel
 1182 of Fig. 2. A held-out batch is sampled using Stim [73], producing a binary detector-event matrix $S \in \{0, 1\}^{M \times N_{\text{det}}}$ and
 1183 observable-flip labels $O \in \{0, 1\}^{M \times k}$. Each decoder receives the same detector-event matrix S and returns predicted
 1184 logical observable flips $\hat{O}$. A shot is counted as a logical failure if at least one predicted logical observable differs from
 1185 the Stim label. The resulting block logical error probability is converted to the per-round logical error rate using
 1186 the fidelity-based protocol described in Appendix C.3. This shared-sample protocol removes Monte Carlo sampling
 1187 variance as a confounding factor when comparing the curves in Fig. 2.

Standard PyMatching. We use PyMatching as the standard minimum-weight perfect matching baseline. For each circuit, we first extract a graphlike detector error model using `stim.Circuit.detector_error_model(decompose_errors=True)`. The resulting DEM is loaded with `pymatching.Matching.from_detector_error_model`, and decoding is performed in batch mode with `decode_batch`. This baseline treats the decomposed error mechanisms as independent graphlike edges, following the standard MWPM assumption [25, 27]. It therefore provides the canonical fast matching reference for surface-code decoding, but it does not explicitly exploit the residual correlations introduced when non-graphlike physical faults are decomposed into graphlike components.

Correlated PyMatching. We additionally evaluate the correlated matching mode implemented in PyMatching. The decoder is constructed from the same decomposed Stim DEM, but with `enable_correlations=True` during both graph construction and batch decoding. This mode preserves correlation information between decomposed graphlike components that originate from the same physical fault. Such correlations are particularly relevant under circuit-level depolarizing noise, where a single fault, for example a Y -type data-qubit error, can induce correlated detection events across the effective matching graph. Operationally, Correlated PyMatching performs a two-pass matching procedure: an initial MWPM pass proposes a candidate set of graph edges, after which edges correlated with this candidate solution are reweighted and a second matching pass is executed. We include this decoder as a stronger matching-based baseline that partially accounts for circuit-level error correlations while remaining within the MWPM decoding paradigm.

Tesseract. Finally, we include Tesseract as an accuracy-oriented search-based reference [76]. In contrast to MWPM-based decoders, Tesseract performs a most-likely-error search over error subsets and returns a low-cost error configuration consistent with the observed detector events. We provide Tesseract with the same detector error model and the same held-out detector-event samples used by the other baselines, and evaluate its predicted logical observable flips against the same Stim-generated observable labels. Because this search-based decoder is substantially more computationally demanding than matching-based decoders, we use it primarily as a high-accuracy reference rather than as a throughput or latency baseline.

All three baselines are training-free: their decoder data structures are constructed solely from the circuit-level noise model, and no neural training samples are used to tune their parameters. For the detector-error-binned analysis in Fig. 2b, the detector-error count is computed from the sampled detector-event vector before decoding, and the accuracy of each decoder is aggregated over the same bins. Thus, differences between the reported baselines and NTU-Transformer reflect decoder behavior rather than differences in sample generation, binning, or evaluation protocol.

3. BB Codes

a. Parameters Setting For BB codes, the training protocol is adapted to the quasi-cyclic, non-graphlike detector-error structure of the syndrome-extraction circuit. For the NTU-Transformer, we do not employ the classical distillation strategy used in the surface-code experiments, since the MWPM-based process teacher used for surface codes is not naturally available for the non-graphlike BB detector-error model. The distillation coefficient is therefore fixed to $\beta = 0$, and the model is supervised only by the final logical observable-flip labels sampled together with detector events using `stim.Circuit.compile_detector_sampler()`. Stabilizer dropout is also disabled, with $p_{\text{mask}} = p_{\text{drop}} = 0$, to avoid introducing additional corruption under sparse final-label supervision. Detector events are generated online with Stim [73] using a custom BB syndrome-extraction circuit following the CNOT-layer schedule of Ref. [15]. A single physical error rate p is applied consistently to Clifford depolarization, reset flips, measurement flips, and pre-round data depolarization, matching the circuit-level depolarizing noise model used for the BB baselines. Optimization uses AdamW with `bfloat16` mixed precision, a 100-step linear warmup, cosine learning-rate decay to 5% of the peak value, and global gradient clipping with maximum norm 1.0. The decoupled weight-decay strategy used for the surface-code Transformer is retained: structural projection weights are assigned weight decay 10^{-2} , while biases, normalization parameters, topological embeddings, logical-query tokens, and readout heads are exempted. Transfer from [[72, 12, 6]] to [[144, 12, 12]] is performed by loading all shape-compatible learnable parameters, regenerating the non-learnable code-dependent topology buffers, and restarting the optimizer schedule from step zero; the geometry-aware RoPE base frequency $\Theta = 100$ is retained across both codes.

TABLE III. Experimental parameter configuration for the NTU-Transformer on BB codes. The from-scratch [[144, 12, 12]] baseline uses the same configuration but extends the maximum training horizon to 50,000 steps.

Code	Rounds (r)	Initial Learning Rate	p_{train}	p_{eval}	Local Batch Size	Grad. Accum.	Max Steps
[[72, 12, 6]]	6	5×10^{-4}	0.005	0.005	512	1	50,000
[[144, 12, 12]]	12	5×10^{-4}	0.005	0.005	256	4	30,000

For NTU-NeuralBP, training is performed at the physical error-inference level rather than the logical-classification level. For each BB code, the Stim detector-error model is converted into a sparse bipartite parity-check representation (H, L) , where H maps elementary error mechanisms to detector syndromes and L maps them to logical observables. During training, an error-mechanism vector e is sampled according to the probabilities specified by the detector-error model, the syndrome is computed as $s = He \bmod 2$, and the network is trained to infer e from s . The objective combines a Focal loss with $\alpha = 0.95$ and $\gamma = 2.0$ to address the sparsity of error mechanisms, together with a syndrome-consistency loss weighted by $\lambda_{\text{syn}} = 0.2$. The BP unrolling depth is fixed to $N_{\text{iter}} = 2d$. Optimization uses AdamW in full float32, a 100-step linear warmup, cosine decay to 5% of the peak learning rate, global gradient clipping at maximum norm 1.0, and a uniform weight decay of 10^{-4} on all trainable parameters. No edge-level or message-level dropout is applied. Transfer from $[[72, 12, 6]]$ to $[[144, 12, 12]]$ is implemented by directly reloading the full set of trainable message-passing parameters and replacing only the sparse graph tensors associated with the target parity-check matrix. Evaluation for both BB neural decoders follows the per-logical-qubit, per-round LER protocol described in Appendix E 1.

TABLE IV. Experimental parameter configuration for the NTU-NeuralBP on BB codes. The BP unrolling depth is fixed to $N_{\text{iter}} = 2d$ for each code.

Code	BP Iter. (N_{iter})	Initial Learning Rate	p_{train}	p_{eval}	Local Batch Size	Grad. Accum.	Max Steps
$[[72, 12, 6]]$	12	3×10^{-4}	0.005	0.005	512	1	15,000
$[[144, 12, 12]]$	24	3×10^{-4}	0.005	0.005	512	1	15,000

b. Baseline Methods We benchmark the NTU decoders against two training-free baseline methods on BB codes: BP+OSD and RelayBP, with the latter evaluated at two per-relay iteration depths (denoted RelayBP- $2d$ and RelayBP- $4d$ in the figure legends). These baseline configurations span the accuracy-complexity spectrum relevant to circuit-level qLDPC decoding, from belief propagation augmented by ordered-statistics post-processing to a memory-assisted belief-propagation heuristic evaluated at depth-aligned and extended per-relay iteration settings.

Common evaluation protocol. All baseline configurations are evaluated on the same held-out detector-event samples generated by the Stim-driven BB syndrome-extraction circuit used throughout this work. A sampled batch yields detector events $S \in \{0, 1\}^{N_{\text{shot}} \times N_{\text{det}}}$ and observable-flip labels $O \in \{0, 1\}^{N_{\text{shot}} \times k}$ with $k = 12$. For decoders operating on the detector-error model, the model is converted into a sparse parity-check representation (H, L) , where H maps elementary error mechanisms to detector syndromes and L maps them to logical observable flips. Given a detector-event vector s , each decoder returns an estimated error-mechanism vector $\hat{e}$, which is converted to predicted logical observable flips as $\hat{O} = L\hat{e} \bmod 2$. A shot is counted as a logical failure if at least one predicted logical observable differs from the Stim-generated observable-flip label. The resulting block logical error probability is converted to the per-round, per-logical-qubit LER using the fidelity-based protocol described in Appendix E 1.

BP+OSD. We use BP+OSD as a standard classical decoder for qLDPC codes [43, 77]. The Stim detector-error model is converted into a sparse bipartite parity-check matrix H together with the corresponding per-mechanism priors, and the decoder is instantiated using `ldpc.BpOsdDecoder` in sum-product mode with combination-sweep ordered-statistics decoding (`osd_method="osd_cs"`, `osd_order=10`). Belief propagation first produces soft reliability estimates on the Tanner graph of H . When the BP estimate does not yield a syndrome-consistent correction, the OSD stage performs an ordered-statistics search over unreliable variables to identify an error configuration consistent with the observed syndrome. This post-processing step partially mitigates the convergence failures caused by short cycles and degeneracy in qLDPC Tanner graphs, making BP+OSD a widely used algorithmic reference for BB-code decoding.

RelayBP. We additionally evaluate RelayBP, a recent BP-based heuristic that augments message passing with a memory mechanism designed to escape trapping-set failures in quantum LDPC decoding [78]. At the start of each relay round, a stochastic memory factor γ is assigned to each check node from a distribution that allows negative values. This memory perturbation helps break degenerate message symmetries that can cause standard BP to stall on BB Tanner graphs. The BP updates are then performed with this memory-modified message schedule until either a syndrome-consistent error configuration is found or the prescribed per-relay iteration depth is reached. We use the released implementation accompanying Ref. [78] with its default memory-factor distribution, and report two evaluation configurations, RelayBP- $2d$ and RelayBP- $4d$, corresponding to per-relay-round BP depths of $2d$ and $4d$ iterations, respectively. The RelayBP- $2d$ configuration is depth-aligned with the NTU-NeuralBP unrolling depth, while RelayBP- $4d$ provides a higher-complexity reference for the accuracy-complexity trade-off. These labels refer to the per-relay-round BP depth rather than the total number of BP updates, because multiple relay rounds may be attempted before termination.

All baseline configurations are training-free: their decoder data structures and priors are constructed solely from the Stim-derived circuit-level detector-error model, and no neural training samples are used to tune their internal parameters. Because all methods are evaluated on the same held-out detector-event samples and observable labels,

differences between the reported baselines and the NTU decoders reflect decoder behavior rather than asymmetries in sample generation or evaluation protocol.

Appendix F: Extended Experimental Results

1. Training Dynamics of NTU-Transformer on Surface Code

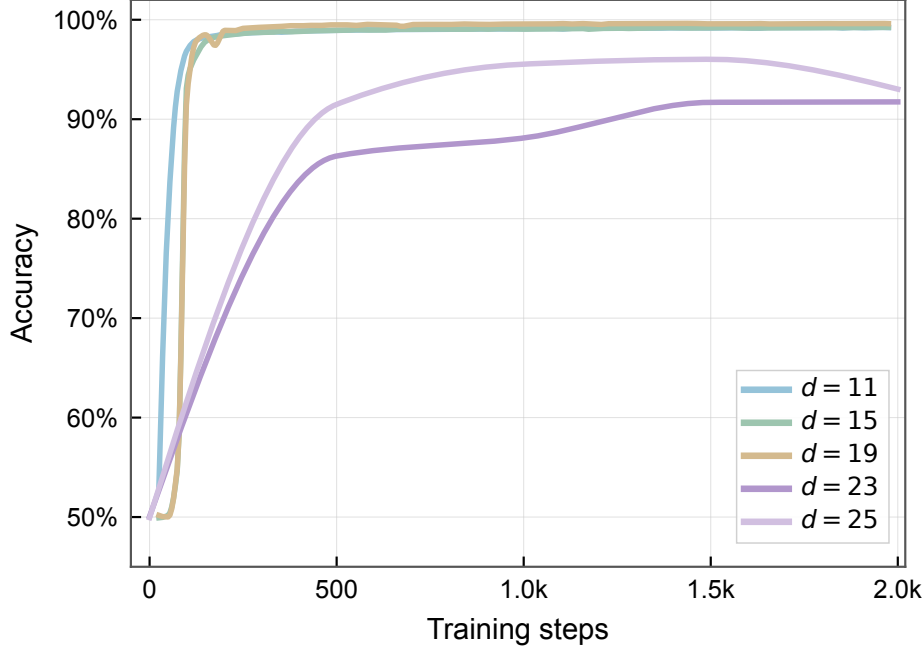


FIG. 4. Extended fine-tuning dynamics of transferred NTU-Transformer models on rotated surface codes. Validation accuracy is plotted against training steps for $d \in \{11, 15, 19, 23, 25\}$. The $d = 11, 15, 19$ runs use $p_{\text{train}} = p_{\text{eval}} = 5 \times 10^{-3}$, while the two largest-distance runs, $d = 23, 25$, use the harder setting $p_{\text{train}} = p_{\text{eval}} = 7 \times 10^{-3}$. The slower rise and lower saturation of the large-distance curves, especially at $d = 23$, therefore reflect the transition to a denser and more challenging noise distribution rather than a failure of transfer. Across all tested distances, the transferred models avoid the random-guessing cold-start plateau near 50%.

Figure 4 extends the main-text training-dynamics study to larger surface-code distances. For $d = 11, 15, 19$, transfer initialization leads to rapid liftoff from the 50% random-guessing regime, consistent with the behavior reported in Fig. 2c. For $d = 23, 25$, we increase the training and validation physical error rate to 7×10^{-3} to obtain more informative nontrivial error configurations within the fixed online sampling budget. This deliberately harder setting explains the slower convergence and lower terminal accuracy of the largest-distance curves. The relevant observation is therefore not that all distances reach the same final accuracy under different noise rates, but that topological transfer consistently places the model on a nontrivial optimization trajectory beyond the cold-start basin.

a. Mode-collapse diagnostic. To verify that the cold-start plateau corresponds to a trivial constant-output mode, we further track whether the scratch-initialized decoder predicts the same logical value for all validation samples. At checkpoint s , after thresholding the output logits, we compute

$$q_b(s) = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} \mathbf{1}[\hat{\mathcal{O}}_s(x) = b], \quad b \in \{0, 1\},$$

where $\mathcal{B}$ is the held-out validation batch. A checkpoint is marked as all-0 collapse if $q_0(s) = 1$, all-1 collapse if $q_1(s) = 1$, and healthy otherwise.

Figure 5 explains the origin of the scratch-training plateau observed in Fig. 2c. As the distance increases, the randomly initialized global readout increasingly falls into degenerate constant-output states, alternating between all-0 and all-1 predictions instead of learning a meaningful syndrome-to-logical mapping. The transfer initialization avoids this failure mode by providing a pre-aligned local perception backbone, leaving fine-tuning to recalibrate the enlarged global decision layer rather than escaping a trivial constant predictor.



FIG. 5. Mode-collapse diagnostic for scratch-initialized NTU-Transformer training on rotated surface codes. Each checkpoint is categorized as all-0 collapse, all-1 collapse, or healthy according to whether the decoder predicts a constant logical value over the validation batch. For $d = 11$, collapse appears only briefly in the early stage; for $d = 15$, collapse occurs intermittently before the model stabilizes; for $d = 19$, all-0 and all-1 collapse persist throughout much of the 2,000-step window. This confirms that the scratch-training plateau is a genuine constant-prediction failure mode rather than merely slow accuracy improvement.

2. Ablation for NTU-Transformer

a. Surface-code ablation. We first perform the architecture ablation on the rotated surface code, where the local planar geometry makes the two structural transfer channels especially transparent (Fig. 6). The transfer task initializes a $d = 11$ target decoder from a pretrained $d = 7$ NTU-Transformer under circuit-level depolarizing noise with $p_{\text{train}} = p_{\text{eval}} = 5 \times 10^{-3}$. We focus on the first 2,000 target fine-tuning steps, so the comparison should be interpreted as a transfer-efficiency test rather than a comparison of ultimate trainability given unlimited optimization.

As shown in Fig. 6, the Full model exhibits the fastest lift-off from the random-guess baseline, showing that physical-coordinate RoPE and the local one-ring stem together provide a strongly aligned initialization for the larger lattice. Ablating either component slows this early fine-tuning process. Removing RoPE weakens the relative geometric phase information in the spatial attention maps, while removing the local stem weakens the consistency between the discrete input embedding vocabulary and the same physical surface-code neighborhood after scaling. Nevertheless, either structural channel alone is sufficient to retain a nontrivial transfer trajectory on the planar surface code. Only the double ablation remains close to random guessing over the fixed early horizon, indicating that when both physical alignment channels are removed, the transferred checkpoint no longer provides a measurable early acceleration over the cold-start regime.

Thus, for the surface code, RoPE and the local stem should be understood primarily as transfer accelerators rather than strict determinants of final model capacity. Their role is to preserve scale-invariant local physical priors so that target-side fine-tuning starts from a nontrivial basin instead of spending substantial optimization compute escaping the cold-start plateau.

b. BB-code ablation. We next perform the same type of architecture ablation on BB-code transfer, where the quasi-cyclic non-local topology makes the consistency of the structural alignment more stringent. To isolate the contribution of the two scale-invariant structural components of the NTU-Transformer—the geometry-aware Rotary Positional Encoding and the polynomial-induced topological embedding—we conduct a 2×2 ablation on the $[[72, 12, 6]] \rightarrow [[144, 12, 12]]$ transfer task (Fig. 7). All four runs share the optimization protocol of Appendix E 3 and are trained for 5,000 fine-tuning steps, which is sufficient to discriminate sustained lift-off from the cold-start basin.

The two ablated implementations are defined as follows. The *code-size-scaled torus RoPE* parameterizes the inverse frequencies as $\text{inv_freq}_i \propto 2\pi/l$ along the first torus axis and analogously $\text{inv_freq}_j \propto 2\pi/m$ along the second. Placing the code-size constants l and m in the denominator causes the same relative offset Δi to produce different RoPE phases on $[[72, 12, 6]]$ ($l = 6$) and $[[144, 12, 12]]$ ($l = 12$). The Full model instead uses the Cartesian, code-size-independent RoPE introduced in Appendix C, $\text{inv_freq}_k = \Theta^{-k/\text{quarter_dim}}$ with $\Theta = 100$, whose inverse-frequency buffer is identical across both codes.

The *torus L_2 -nearest embedding* replaces the polynomial-induced offset set of Appendix C—which yields $K_{\text{same}} = 12$ same-type and $K_{\text{cross}} = 9$ cross-type neighbors derived from the off-diagonal terms of AA^T , BB^T , and AB —with a geometric heuristic that selects each stabilizer’s $K = 6$ torus- L_2 -nearest same-type neighbors. The resulting neighbor set includes pairs that share no common data qubit on the BB Tanner graph while omitting genuine polynomial-induced neighbors at larger L_2 distance, so the same embedding index no longer encodes the same physical relation across the two code sizes.

In contrast to the surface-code case, breaking either alignment is sufficient to remove the measurable early transfer

Per-qubit accuracy

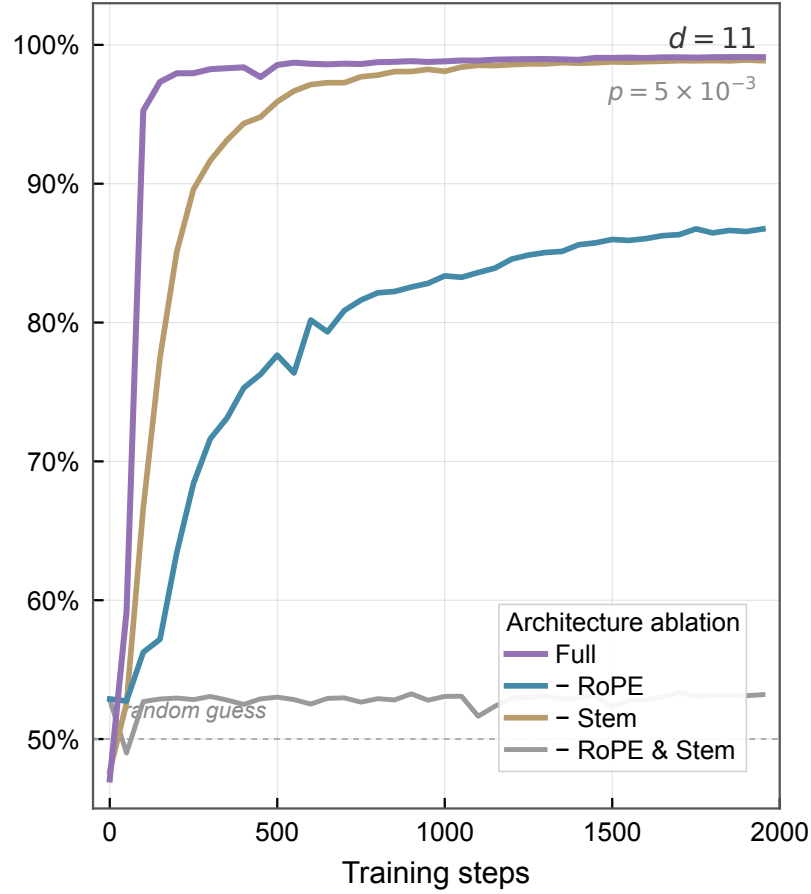


FIG. 6. Architecture ablation of the NTU-Transformer on rotated surface-code transfer from $d = 7$ to $d = 11$ at $p = 5 \times 10^{-3}$. Accuracy is plotted against target fine-tuning steps for four transfer configurations. The *Full* model uses physical-coordinate RoPE together with the local one-ring stem. The *RoPE ablation* removes coordinate-dependent phase information from self-attention. The *Stem ablation* replaces the distance-invariant one-ring stencil by a scaled odd-ring stencil, weakening the alignment between a discrete embedding index and the same physical neighborhood across distances. The *Both ablated* configuration removes both structural alignment channels. Single-component ablations delay early lift-off, while removing both components leaves the model close to the random-guess regime within this 2,000-step horizon.

1346 advantage on BB codes. The RoPE ablated, embedding ablated, and double-ablated configurations all remain close to
 1347 the 50% random-guess plateau within the 5,000-step fine-tuning horizon, while only the Full model exhibits sustained
 1348 lift-off. This does not imply that the target architectures are untrainable given enough optimization compute; rather,
 1349 it shows that the specific cross-code transfer acceleration provided by NTU depends on preserving both the positional
 1350 phase alignment and the polynomial neighborhood alignment in this non-local quasi-cyclic topology.

1351 3. Transfer from Partially Trained Source Checkpoints

1352 A practical question for the NTU transfer pipeline is whether the source model must be fully converged before it
 1353 can provide a useful initialization for a larger target code. If transferable structural priors emerge before source-task
 1354 saturation, then the pretraining compute required to initiate cross-distance transfer can be substantially reduced.
 1355 We test this directly on both representative code families by transferring from source checkpoints taken at different
 1356 stages of the same pretraining trajectory, while keeping the target architecture, optimizer schedule, noise setting, and
 1357 fine-tuning protocol fixed.

1358 As shown in Fig. 8, partially trained source checkpoints remain effective for both the rotated surface-code and

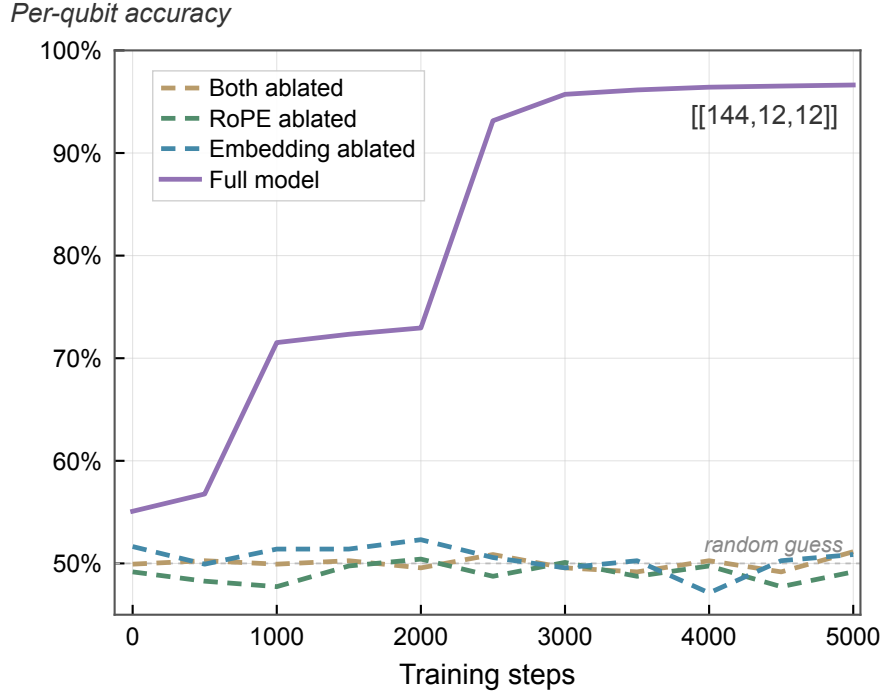


FIG. 7. Ablation of the two scale-invariant structural components of the NTU-Transformer on $[[144, 12, 12]]$ transfer. Per-qubit accuracy is plotted against fine-tuning steps for four conditions arranged on a 2×2 grid: the *Full* model with both the Cartesian, code-size-independent RoPE and the polynomial-induced topological embedding of Appendix C; *RoPE ablated*, which replaces the Cartesian RoPE with a code-size-scaled torus variant; *Embedding ablated*, which replaces the polynomial-induced offsets with a torus L_2 -nearest heuristic; and *Both ablated*. Within the fixed early fine-tuning horizon, only the Full model exhibits sustained transfer lift-off, whereas the three ablated configurations fail to realize a measurable transfer advantage.

BB-code transfers. In the surface-code experiment, a partially trained $d = 7$ source checkpoint transfers successfully to the $d = 11$ target task. In the BB-code experiment, an under-converged $[[72, 12, 6]]$ source checkpoint similarly transfers to the larger $[[144, 12, 12]]$ target task. In both cases, the under-converged source starts from a lower target accuracy than the converged source, indicating that the source representation is not yet fully calibrated. However, this initial deficit does not prevent target-side learning: the fine-tuning curves rise on comparable time scales, avoid the random-guess plateau, and approach similar accuracy ranges after continued training.

These results indicate that NTU transfer does not rely on complete source-task saturation. Instead, the transferable ingredients appear to be the local, scale-invariant topological priors encoded in the perception backbone, which emerge before the source task reaches its final accuracy. Full source convergence mainly improves the initial target calibration, whereas the remaining mismatch introduced by an immature source checkpoint can be absorbed by continued target-side fine-tuning. This further reduces the effective pretraining burden of the NTU framework, since cross-distance transfer can be initiated once a nontrivial local perception backbone has formed rather than only after exhaustive source-task optimization.

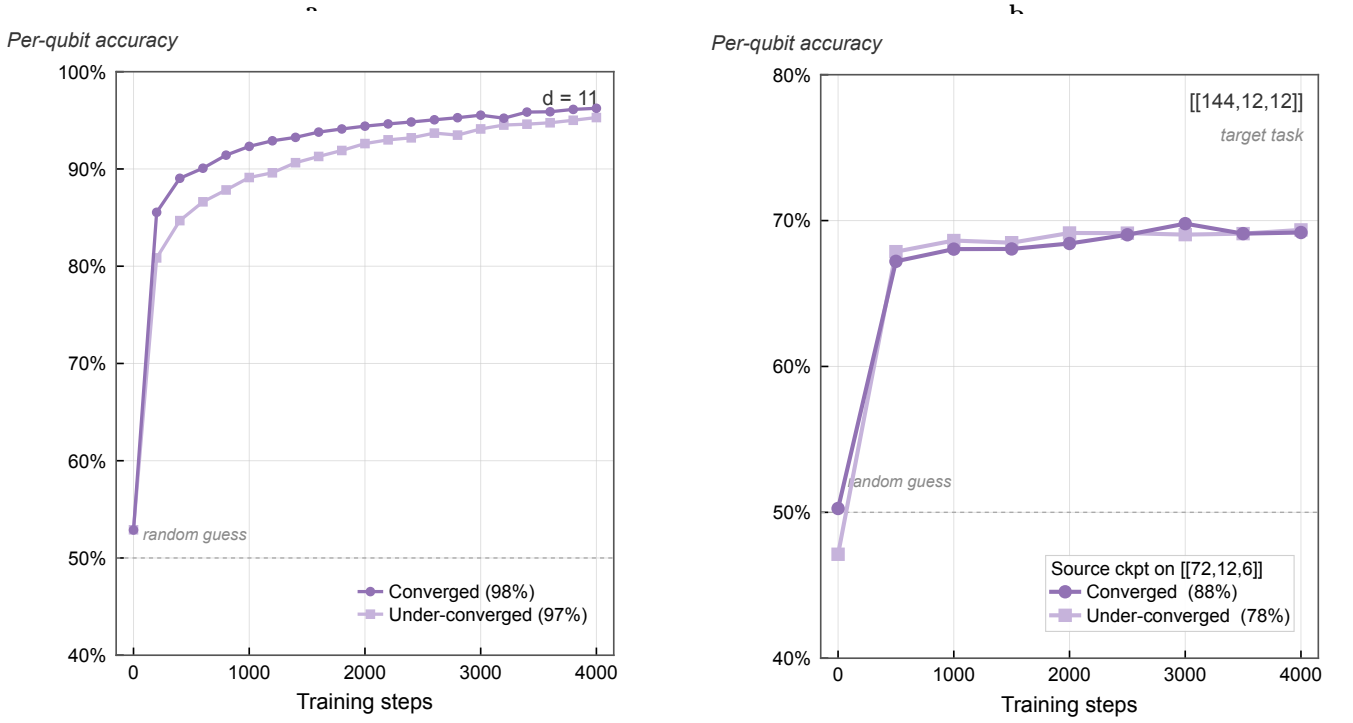


FIG. 8. Transfer from source checkpoints at different training maturities. **a**, Rotated surface-code transfer from a $d = 7$ source checkpoint to the $d = 11$ target task. The converged and under-converged source checkpoints have source-task accuracies of 98% and 97%, respectively. Although the under-converged source gives a lower initial target accuracy, its fine-tuning trajectory rises on a comparable time scale and progressively approaches the converged-source transfer. **b**, BB-code transfer from $[[72, 12, 6]]$ source checkpoints to the $[[144, 12, 12]]$ target task. The converged and under-converged source checkpoints have source-task accuracies of 88% and 78%, respectively. Despite the lower source maturity, both target trajectories escape the 50% random-guess baseline within the early fine-tuning stage and approach the same accuracy range after continued training. Together, these experiments show that effective NTU transfer does not require complete source-task convergence: a partially trained source model can already provide a useful scale-invariant initialization for larger target codes, with the lower initial target accuracy largely compensated by subsequent target-side fine-tuning.